

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA VIỄN THÔNG II

-----***-----



BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN

NĂM HỌC 2025-2026

07-SV-2025-VT2

**THIẾT KẾ VÀ THI CÔNG MÔ HÌNH BÃI GIỮ XE
THÔNG MINH CÓ THỂ THEO DÕI, QUẢN LÝ TỪ XA
QUA THIẾT BỊ DI ĐỘNG**

Thuộc nhóm ngành: Công Nghệ Thông Tin, Viễn Thông

TP. HCM, 10/2025

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA VIỄN THÔNG II

-----***-----



BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN

NĂM HỌC 2025-2026

07-SV-2025-VT2

**THIẾT KẾ VÀ THI CÔNG MÔ HÌNH BÃI GIỮ XE THÔNG MINH CÓ
THẺ THEO DÕI, QUẢN LÝ TỪ XA QUA THIẾT BỊ DI ĐỘNG**

Sinh viên thực hiện: **Phan Văn Tuấn Anh** Nam, Nữ: Nam

Lớp, khoa: D23CQVT01-N, Khoa Viễn Thông 2 Năm thứ: 3/4,5 năm

Sinh viên thực hiện: **Nguyễn Thiết Chân** Nam, Nữ: Nam

Lớp, khoa: D23CQCI01-N, Khoa Viễn Thông 2 Năm thứ: 3/4,5 năm

Sinh viên thực hiện: **Trương Phan Nhân** Nam, Nữ: Nam

Lớp, khoa: D23CQCN01-N, Khoa Công Nghệ Thông Tin 2 Năm thứ: 3/4,5 năm

Sinh viên thực hiện: **Huỳnh Thị Nguyệt Nhi** Nam, Nữ: Nữ

Lớp, khoa: D23CQVT01-N, Khoa Viễn Thông 2 Năm thứ: 3/4,5 năm

Sinh viên thực hiện: **Nguyễn Thị Bảo Tuyền** Nam, Nữ: Nữ

Lớp, khoa: D23CQCI01-N, Khoa Viễn Thông 2 Năm thứ: 3/4,5 năm

Người hướng dẫn: **ThS. PHẠM QUỐC HỢP**

MỤC LỤC

LỜI CẢM ƠN	5
DANH MỤC HÌNH	6
DANH MỤC BẢNG	8
CHƯƠNG 1: GIỚI THIỆU	10
1.1. Lý do chọn đề tài	10
1.2. Sản phẩm tương tự thực tế	10
1.3. Mục tiêu của đề tài.....	12
1.4. Sơ lược về bố cục, nội dung cuốn báo cáo	12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	13
2.1. Tổng quan về IoT.....	13
2.1.1. Giới thiệu.....	13
2.1.2. Ứng dụng của IoT trong bãi giữ xe thông minh (Smart Parking)	15
2.2. Vi điều khiển và các linh kiện được sử dụng trong mô hình bãi giữ xe thông minh [7]	16
2.2.1. Vi điều khiển ESP32 và Arduino Uno R3	16
2.2.2. Cảm biến hồng ngoại (LM393)	18
2.2.3. RFID – RC522	19
2.2.4. Camera ESP32 – CAM	20
2.2.5. Các linh kiện khác.....	21
2.3. Các chuẩn giao tiếp và giao thức truyền thông được sử dụng trong mô hình bãi giữ xe thông minh	23
2.3.1. Chuẩn giao tiếp SPI [8].....	23
2.3.2. Chuẩn giao tiếp I2C [9]	24
2.3.3. Chuẩn giao tiếp UART [10].....	25
2.3.4. Giao thức WebSocket [11].....	26
2.3.5. Giao thức HTTP [12]	27
CHƯƠNG 3: THIẾT KẾ MÔ HÌNH BÃI GIỮ XE THÔNG MINH.....	28
3.1. Yêu cầu đối với mô hình bãi giữ xe thông minh được thiết kế.....	28

3.1.2. Tính năng của các thành phần trong hệ thống.....	29
3.1.3. Các trường hợp sử dụng.....	29
3.2. Thiết kế phần cứng	31
3.2.1. Sơ đồ khối của hệ thống.....	31
3.2.2. Sơ đồ nối các linh kiện	33
3.3. Thiết kế phần mềm	35
3.3.1. Lưu đồ hoạt động của hệ thống	35
3.3.2. Chương trình phần mềm	36
3.3.3. Nhiệm vụ, chức năng và ý nghĩa của các phần mềm được sử dụng ..	45
CHƯƠNG 4: KẾT QUẢ THỰC HIỆN VÀ KIỂM THỬ	48
4.1. Kết quả thực hiện.....	48
4.1.1. Phần cứng	48
4.1.2. Phần mềm	49
4.2. Kiểm thử	53
4.3. Nhận xét	53
CHƯƠNG 5: KẾT LUẬN.....	54
TÀI LIỆU THAM KHẢO.....	55
PHỤ LỤC CODE.....	56

LỜI CẢM ƠN

Đầu tiên, nhóm xin bày tỏ lòng biết ơn chân thành đến các thầy cô trong Khoa Viễn Thông II nói riêng cũng như trong Học viện Công nghệ Bưu chính Viễn thông cơ sở tại TP. HCM nói chung đã tạo điều kiện thuận lợi để chúng em có cơ hội thực hiện đề tài nghiên cứu khoa học này. Đây không chỉ là một hành trình học tập đầy thử thách và trải nghiệm mà còn là cơ hội quý báu để nhóm chúng em có thêm kinh nghiệm về các kỹ năng và tinh thần làm việc nhóm.

Trong quá trình thực hiện đề tài, nhóm gửi lời cảm ơn sâu sắc đến giảng viên hướng dẫn, người đã tận tâm đồng hành, chia sẻ kinh nghiệm và định hướng cho chúng em. Sự hỗ trợ của thầy cũng góp phần vào động lực để chúng em hoàn thiện và phát triển đề tài.

Bên cạnh đó, cảm ơn sự đồng hành và đóng góp ý kiến của các thành viên trong nhóm. Mọi cố gắng và sự hợp tác của từng thành viên đã tạo nên kết quả chung hôm nay. Cuối cùng, đây là một hành trình đầy khó khăn và ý nghĩa nhưng cũng vô cùng đáng nhớ vì có sự đồng hành, hỗ trợ và giúp đỡ của tất cả mọi người.

Xin chân thành cảm ơn!

Thành Phố Hồ Chí Minh, ngày ... tháng ... năm 2025

TM. Nhóm sinh viên thực hiện

Phan Văn Tuấn Anh

DANH MỤC HÌNH

Hình 2.1: Các thành phần của hệ thống IoT

Hình 2.2: Kiến trúc IoT năm lớp

Hình 2.3: Kiến trúc IoT chi tiết

Hình 2.4: Ứng dụng trong IoT

Hình 2.5: Vi điều khiển ESP32 32pin

Hình 2.6: Arduino Uno R3

Hình 2.7: Cảm biến hồng ngoại (IR sensor – LM393)

Hình 2.8: Nguyên ý hoạt động của IR sensor

Hình 2.9: Màn hình LCD 16x2

Hình 2.10: RFID – RC522

Hình 2.11: ESP32 – CAM - MB

Hình 2.12: Micro servo 9g -SG90

Hình 2.13: Chuẩn giao tiếp SPI

Hình 2.14: Sơ đồ kết nối giao tiếp SPI giữa một thiết bị Master (vi điều khiển) và nhiều thiết bị Slave (cảm biến, các linh kiện khác).

Hình 2.15: Sơ đồ kết nối giao tiếp I2C giữa một thiết bị Master và nhiều thiết bị Slave

Hình 2.16: Nguyên lý hoạt động của chuẩn giao tiếp UART

Hình 2.17: Nguyên lý hoạt động của giao thức WebSocket

Hình 2.18: Nguyên lý hoạt động của giao thức HTTP

Hình 3.1: Sơ đồ khối tổng quát của hệ thống.

Hình 3.2: Sơ đồ khối chi tiết của hệ thống.

Hình 3.3: Sơ đồ nối các linh kiện

Hình 3.4: Lưu đồ hoạt động của hệ thống

Hình 3.5: Sơ đồ khối phần mềm cho vi điều khiển

Hình 3.6: Giao diện phần mềm Arduino IDE

Hình 3.7: Giao diện phần mềm Visual Studio Code

Hình 3.8: Hướng dẫn Node.js trong Visual Studio Code

Hình 3.9: Hướng dẫn MongoDB trong Visual Studio Code

Hình 4.1: Mô hình phân cứng

Hình 4.2: Quét thẻ RFID

Hình 4.3: Camera chụp biển số xe

Hình 4.4: Rào chắn mở cho xe vào bãi

Hình 4.7: Giao diện đăng ký của khách hàng

Hình 4.8: Giao diện đăng nhập của khách hàng

Hình 4.9: Giao diện đăng nhập thành công

Hình 4.10: Giao diện đặt chỗ

Hình 4.11: Giao diện đặt chỗ thành công

Hình 4.12: Vé đặt chỗ thành công

Hình 4.13: Tra cứu thông tin bằng mã vào cổng

Hình 4.14: Giao diện của nhà quản lý

Hình 4.15: Chụp ảnh biển số khi xe vào bãi thành công

Hình 4.16: Biển số xe đã được lưu vào lịch sử truy cập

Hình 4.17: Chụp ảnh biển số khi xe rời bãi thành công

Hình 4.18: Biển số xe vào và rời bãi đã khớp nhau

Hình 4.19: Chụp ảnh biển số khi xe rời bãi thất bại

Hình 4.20: Biển số xe vào và rời bãi không khớp nhau

DANH MỤC BẢNG

Bảng 3.1: Trường hợp ứng dụng thực tế (user case 1)

Bảng 3.2: Trường hợp ứng dụng thực tế (user case 2)

Bảng 3.3: Trường hợp ứng dụng thực tế (user case 3)

Bảng 3.4: Thư viện được sử dụng trong esp32.ino

Bảng 3.5: Thư viện được sử dụng trong arduino.ino

Bảng 3.6: Thư viện được sử dụng trong esp32_cam.ino

Bảng 3.7: Chức năng chung của camService.js

Bảng 3.8: Chức năng chung của booking.js

Bảng 3.9: Chức năng chung của user.js

Bảng 3.10: Chức năng chung của confirm.js

Bảng 3.11: Chức năng chung của manager.js

TỪ VIẾT TẮT

Từ viết tắt	Chữ viết đầy đủ
MCU	Microcontroller Unit
ESP32	Espressif System on Chip
IR sensor	Infrared Sensor
LCD	Liquid Crystal Display
RFID	Radio Frequency Identification
SPI	Serial Peripheral Interface
UART	Universal Asynchronous Receiver/Transmitter
I2C	Inter-Integrated Circuit
PWM	Pulse Width Modulation
GPIO	General Purpose Input/Output
HTTP	Hypertext Transfer Protocol
API	Application Programming Interface
CAM	Camera Module
USB	Universal Serial Bus
IDE	Integrated Development Environment
IoT	Internet of Things

CHƯƠNG 1: GIỚI THIỆU

1.1. Lý do chọn đề tài

Hiện nay, ở các đô thị lớn như TP. HCM, Hà Nội, mật độ về phương tiện giao thông ngày càng tăng nhanh chóng khiến việc tìm chỗ giữ xe trở nên khó khăn hơn, mất nhiều thời gian, gây ùn tắc giao thông và ô nhiễm[1]. Nhu cầu giữ xe không chỉ là chỗ để tạm mà còn là sự thuận tiện, an toàn, minh bạch về cả chi phí và thời gian.

Sự phát triển của CNTT và IoT đã mở ra một hướng mới cho việc xây dựng bãi giữ xe thông minh. Nhờ vào việc tích hợp các cảm biến và phần mềm, hệ thống sẽ giúp người dùng tìm, đặt trước, giữ và thanh toán dễ dàng hơn.

Việc ứng dụng CNTT và IoT vào bãi giữ xe giúp giảm thời gian tìm chỗ, nâng cao hiệu quả quản lý, giảm nhân lực, tăng tính minh bạch trong quá trình gửi - nhận xe, đồng thời cập nhật trạng thái thời gian thực, giúp người dùng chủ động lựa chọn vị trí phù hợp.

Đề tài phù hợp xu hướng chuyển đổi số, ứng dụng IoT, kết nối thiết bị phần cứng (Arduino, ESP32, cảm biến, RFID, LCD...) với phần mềm quản lý (web/app), qua đó tạo điều kiện cho sinh viên tiếp cận thực tế, học hỏi kỹ năng thiết kế, lập trình, hệ thống tích hợp. Bên cạnh đó, bài nghiên cứu khoa học này cũng giúp rèn luyện khả năng làm việc nhóm, tư duy sáng tạo, giải quyết vấn đề thực tiễn, đáp ứng yêu cầu đổi mới sáng tạo trong giáo dục và xã hội.

1.2 Sản phẩm tương tự thực tế

❖ Tìm hiểu

Nhiều sản phẩm và mô hình miễn phí xe thông minh đã được phát triển khai thực tế tại Việt Nam và trên thế giới với các đặc điểm nổi bật: Các bãi đỗ xe thông minh tại Hà Nội, TP.HCM, Đà Nẵng, trung tâm thương mại, bệnh viện, chung cư đã ứng dụng công nghệ cảm biến, camera nhận diện biển số, ...

➤ Một số hệ thống nổi bật :

- Bãi giữ xe thông minh *VinFast (Hà Nội, TP.HCM)* [2]

Hệ thống ứng dụng cảm biến, camera nhận diện biển số và phần mềm quản lý tập trung. Tích hợp thanh toán điện tử không tiền mặt qua app, theo dõi trạng thái bãi đỗ thời gian thực. Bãi xe di chuyển bằng thang nâng hoặc dạng tháp, có sức chứa lớn, tốc độ lấy xe nhanh (1-2 phút). Phù hợp các tòa nhà cao tầng hoặc khu đô thị đông đúc.

- Hệ thống bãi giữ xe tự động *Puzzle Parking (TP.HCM)* [3]

Hệ thống giữ xe xếp hình (Puzzle Parking) theo nhiều tầng, sử dụng pallet tự động di chuyển xe. Điều khiển bằng bảng điều khiển cảm ứng hoặc màn hình, có mật mã bảo vệ an toàn. Thời gian di chuyển xe nhanh, khoảng 1-2 phút/lượt,

phù hợp xếp đặt tối ưu không gian. Phù hợp khu trung tâm thương mại, chung cư cao tầng.

- Bãi giữ xe thông minh *Green Smart (Việt Nam)* [4]

Sử dụng cảm biến siêu âm, RFID kiểm soát xe ra vào cùng phần mềm quản lý hiện đại. Hệ thống cho phép giám sát, cảnh báo và quản lý doanh thu tự động. Đáp ứng nhu cầu bãi xe vừa và nhỏ, như chung cư, trường học, văn phòng.

❖ Nhận xét và đánh giá:

Hiện nay, nhiều hệ thống bãi giữ xe thông minh tại Việt Nam đã áp dụng thành công các công nghệ tiên tiến nhằm xử lý vấn đề thiếu chỗ giữ xe ở các đô thị lớn. Ví dụ, hệ thống bãi giữ xe của *VinFast ở Hà Nội và Puzzle Parking TP.HCM* có quy mô lớn, sử dụng các cảm biến, camera để nhận biết biển số xe kết hợp với phần mềm quản lý tập trung. Hệ thống này không chỉ giúp điều phối xe ra vào hiệu quả mà còn giám sát chặt chẽ 24/7, đồng thời hỗ trợ thanh toán không dùng tiền mặt qua ứng dụng di động, góp phần cải thiện trải nghiệm cho người sử dụng. Tuy nhiên những hệ thống bãi giữ xe này còn một số nhược điểm đáng chú ý:

➤ Bãi đậu xe VinFast (Hà Nội, TP.HCM):

- Chi phí đầu tư xây dựng rất lớn để ứng dụng công nghệ tháp xe, nâng cao và camera nhận diện biển số hiện đại, khó áp dụng cho các bãi nhỏ hơn.
- Phần lớn ưu tiên phát triển ở khu đô thị cao, chưa phù hợp hoặc khó mở rộng sang khu dân cư thường hoặc khu vực có quy mô vừa và nhỏ.
- Chức năng đặt chỗ trước web/app chưa phổ biến, phải trực tiếp đến bãi kiểm tra chỗ trống, dễ xảy ra ùn quy tắc vào giờ cao điểm.

➤ Bãi đậu xe Puzzle Parking (TP.HCM):

- Hệ thống cơ sở/phần mềm phụ thuộc nhiều vào động cơ, pallet tự động; khi có lỗi kỹ thuật hoặc mất điện thì toàn bộ công việc sẽ bị ảnh hưởng, không thể vận hành thủ công.
- Phí bảo trì, sửa chữa hệ thống cao do phải thường xuyên bảo trì các thiết bị.
- Chủ yếu điều khiển, vận hành tại chỗ, chưa có ứng dụng web/app thông minh để tra cứu, đặt chỗ từ xa; quy mô lớn khó áp dụng cho các bãi nhỏ hoặc nơi giới hạn chiều cao, mặt bằng.

➤ Green Smart (Việt Nam):

- Chủ yếu đáp ứng nhu cầu bãi vừa và nhỏ; chưa phù hợp bãi xe đa tầng, nhiều xe, công suất lớn.

- Tính chính xác và ổn định phụ thuộc vào cảm biến siêu âm, có thể bị ảnh hưởng bởi môi trường thực tế (bùn đất, mưa sẽ làm nhiễu tín hiệu...).

Tóm lại, các sản phẩm bãi giữ xe thông minh hiện nay đóng vai trò rất quan trọng trong việc nâng cao hiệu quả quản lý và giải quyết các vấn đề về giao thông đô thị. Đề tài nghiên cứu với việc bổ sung phần mềm đặt chỗ trước cùng việc tích hợp các thiết bị phần cứng phù hợp đang hướng đến một giải pháp linh hoạt, tiết kiệm chi phí hơn, giúp cải thiện trải nghiệm người dùng một cách toàn diện.

1.3 Mục tiêu của đề tài

- Thiết kế bãi giữ xe thông minh được ứng dụng công nghệ IoT để có thể theo dõi, quản lý từ xa thông qua Internet.
- Thi công mô hình bãi giữ xe theo thiết kế.
- Xây dựng app trên thiết bị di động giúp người dùng có thể theo dõi và quản lý bãi giữ xe từ xa thông qua Internet.

=> Kết quả mong đợi là mô hình hệ thống bãi giữ xe thông minh với ứng dụng trên web giúp người dùng chủ động tìm và giữ chỗ, biết trước số tiền phải trả, tiết kiệm thời gian, giảm ùn tắc giao thông, đồng thời, hỗ trợ chủ bãi xe quản lý hiệu quả, tự động và minh bạch hơn.

1.4. Sơ lược về bố cục, nội dung cuốn báo cáo

Báo cáo gồm các phần chính như sau:

CHƯƠNG 1: GIỚI THIỆU

Lý do chọn đề tài, sản phẩm tương tự thực tế, mục tiêu và bố cục cuốn báo cáo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Tổng quan về IoT và ứng dụng trong bãi giữ xe thông minh, linh kiện phần cứng, các chuẩn giao tiếp, giao thức truyền thông và các phần mềm sử dụng.

CHƯƠNG 3: THIẾT KẾ MÔ HÌNH BÃI GIỮ XE

Xác định yêu cầu, chức năng của hệ thống. Thiết kế phần cứng và phần mềm.

CHƯƠNG 4: KẾT QUẢ VÀ KIỂM THỬ

Mô hình hoạt động thực tế, giao diện phần mềm, kiểm thử và đánh giá hệ thống.

CHƯƠNG 5: KẾT LUẬN

Tóm tắt toàn bộ nội dung và kết quả đạt được của đề tài. Nêu các hạn chế và hướng phát triển.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về IoT

2.1.1. Giới thiệu

Internet of Things (IoT) là hệ thống kết nối các thiết bị vật lý (như cảm biến, vi điều khiển, điện thoại, camera, xe cộ, máy móc,...) với Internet để thu thập, truyền tải và xử lý dữ liệu. Nhờ IoT, các thiết bị không chỉ hoạt động riêng lẻ mà còn có thể “giao tiếp” với nhau[5].

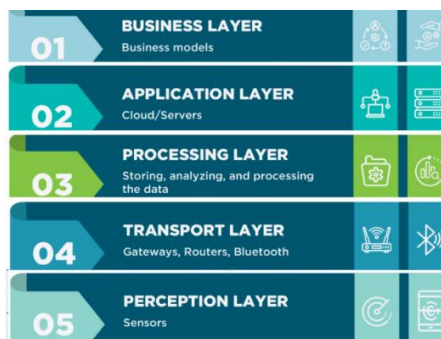
Mục tiêu của IoT là mang khả năng kết nối và điều khiển vào những vật thể vốn “không thông minh”, tạo nên hệ sinh thái tự động hóa, tiết kiệm tài nguyên và hỗ trợ con người hiệu quả hơn. Trong thực tế, IoT giúp các thiết bị tự vận hành hoặc được điều khiển từ xa, góp phần nâng cao chất lượng cuộc sống[5].

❖ **Các thành phần của hệ thống IoT:** [6]



Hình 2.1: Các thành phần của hệ thống IoT

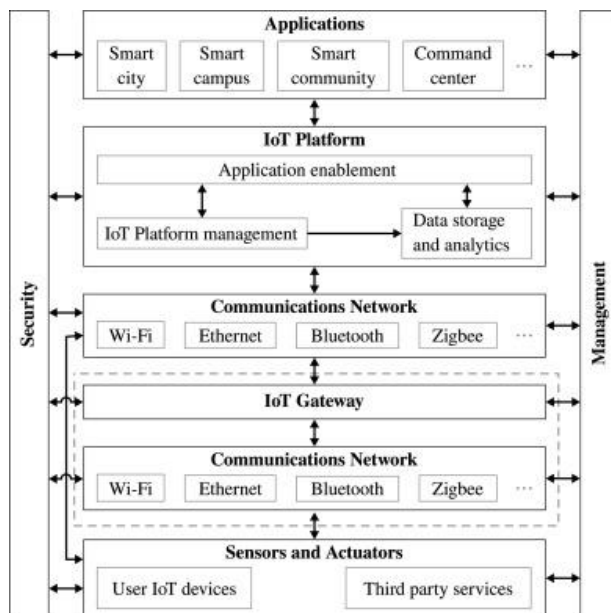
- **Sensor:** Cảm biến có chức năng thu thập dữ liệu từ môi trường và truyền đến lớp xử lý tiếp theo.
- **Gateway:** Giao tiếp trung gian giữa các sensor có công suất thấp và đám mây.
- **Connectivity:** là đường truyền internet để truyền dữ liệu từ thiết bị này sang thiết bị khác hoặc giữa thiết bị và đám mây.
- **Cloud:** Các hệ thống IoT gửi dữ liệu khổng lồ từ các thiết bị và dữ liệu này cần được quản lý hiệu quả để tạo ra đầu ra có ý nghĩa.
- **Analytics:** Quá trình chuyển đổi dữ liệu thô thành một số dạng có ý nghĩa, định dạng dễ hiểu đối với người dùng.
- **User Interface:** Giao diện người dùng là thành phần tiếp xúc và đưa dữ liệu tới người sử dụng các dịch vụ. Đó là một phần của hệ thống tương tác với người dùng cuối.



Hình 2.2: Kiến trúc IoT năm lớp

❖ **Kiến trúc IoT năm lớp:** [6]

- **Lớp cảm nhận (Perception layer)** : thu thập dữ liệu. (còn gọi Sensing)
- **Lớp vận chuyển (Transport layer):** Truyền dữ liệu từ lớp 5 lên lớp 3 (có thể qua Gateway hoặc trực tiếp qua Internet).
- **Lớp ứng dụng (Application layer):** Tương tác với người sử dụng.
- **Lớp xử lý (Processing Layer):** Tăng cường tính năng bảo mật và tương tác với người dùng.
- **Lớp kinh doanh (Business layer):** Tăng cường tính năng thân thiện với người dùng.



Hình 2.3: Kiến trúc IoT chi tiết

- ❖ **Kiến trúc IoT chi tiết:** Cảm biến thu thập dữ liệu → dữ liệu đi qua mạng và cổng IoT → được xử lý, lưu trữ trên nền tảng IoT thông qua mạng truyền thông → hiển thị kết quả lên ứng dụng → người dùng có thể điều khiển ngược lại các thiết bị.



Hình 2.4: Ứng dụng trong IoT

2.1.2. Ứng dụng của IoT trong bãi giữ xe thông minh (Smart Parking)

❖ Bãi giữ xe thông minh là gì?

Bãi giữ xe thông minh nói chung là một hệ thống sử dụng công nghệ để cải thiện quản lý và sử dụng không gian giữ xe. Cụ thể hơn, bãi giữ xe thông minh còn là hệ thống quản lý bãi giữ xe ứng dụng công nghệ tự động hoá, công nghệ thông tin và IoT giúp việc quản lý bãi giữ xe được dễ dàng và tự động.

Bãi giữ xe thông minh mang lại khả năng kiểm soát, chống thất thoát cho chủ, giảm thiểu thời gian tìm kiếm chỗ đỗ, tiết kiệm năng lượng và giảm thiểu lưu lượng giao thông tại các khu vực đô thị. Nó cũng cải thiện trải nghiệm người dùng thông qua tính năng dễ sử dụng và hiệu quả của các web/app và hệ thống quản lý thông minh.

❖ Ứng dụng của IoT trong bãi giữ xe thông minh

▪ Đặt và giữ chỗ trước khi qua bãi

Chức năng đặt và giữ chỗ trước trong bãi giữ xe thông minh ứng dụng công nghệ IoT, cảm biến và nền tảng kết nối để giúp người dùng chủ động chọn chỗ đỗ trước khi đến bãi. Thông qua website, người dùng có thể xem chỗ trống theo thời gian thực, gửi yêu cầu đặt chỗ, và nhận mã. Khi đến nơi, người dùng đưa mã để xác thực, quét RFID, đồng thời camera sẽ chụp, lưu lại biển số xe, sau đó khách hàng nhận lại thẻ từ và barie được mở, giúp quá trình vào bãi nhanh chóng và tiện lợi.

Dữ liệu từ cảm biến được gửi về máy chủ để theo dõi và quản lý trạng thái chỗ đỗ, giúp nhà quản lý tối ưu sử dụng không gian, giảm ùn tắc và dự đoán nhu cầu gửi xe. Nhờ đó, hệ thống không chỉ nâng cao trải nghiệm người dùng mà còn tăng hiệu quả vận hành và hiện đại hóa quản lý bãi giữ xe trong đô thị thông minh.

▪ Quản lý xe ra/vào bãi giữ xe

Quản lý vào ra trong bãi giữ xe thông minh sử dụng các công nghệ như nhận diện biển số xe, đếm số lượng vị trí khả dụng và phân tích dữ liệu để cải thiện hiệu quả hoạt động, từ đó giúp quản lý vị trí còn trống một cách chính xác và nhanh chóng, đồng thời có thể tối ưu hóa sử dụng không gian giữ xe. Nhờ vào các thông tin chi tiết này, các nhà quản lý dễ dàng kiểm soát việc sử dụng không gian giữ xe, cải thiện trải nghiệm người dùng bằng cách giảm thiểu thời gian tìm kiếm chỗ đỗ.

- ***Hiện thị chi phí dự kiến***

Chức năng hiển thị chi phí dự kiến trong bãi giữ xe thông minh giúp người dùng biết trước số tiền cần trả dựa trên thời gian gửi xe. Khi người dùng đặt chỗ trước, hệ thống sẽ ghi nhận thông tin và tính toán chi phí tạm thời. Khi người dùng rời bãi, hệ thống tự động tính toán lại tổng thời gian đỗ thực tế và hiển thị số tiền cần thanh toán trên web và màn hình LCD. Nhờ đó, người dùng có thể chủ động lựa chọn thời gian đỗ phù hợp với ngân sách, đồng thời tránh các chi phí phát sinh ngoài ý muốn. Với nhà quản lý, tính năng này giúp minh bạch hóa quy trình thu phí, giảm sai sót và nâng cao sự hài lòng của khách hàng.

- ***Đóng/mở barrier tự động***

Bãi giữ xe thông minh được tích hợp hệ thống đóng mở barie tự động dựa trên công nghệ nhận diện biển số xe và thẻ RFID trong danh sách cho phép. Khi xe tiến đến cổng, camera nhận diện biển số kết hợp với bộ đọc thẻ RFID sẽ kiểm tra thông tin phương tiện. Nếu biển số hoặc thẻ trùng khớp với dữ liệu đã đăng ký trong hệ thống, barrier sẽ tự động mở để xe ra hoặc vào bãi một cách nhanh chóng và thuận tiện.

Việc kết hợp nhận diện biển số và thẻ RFID giúp tăng độ chính xác và an toàn. Hệ thống còn kết nối với cảm biến và bộ điều khiển thông minh, cho phép nhân viên kiểm soát cổng quản lý giám sát lưu lượng xe theo thời gian thực, ngăn chặn phương tiện không hợp lệ, và điều phối hoạt động ra vào hiệu quả hơn.

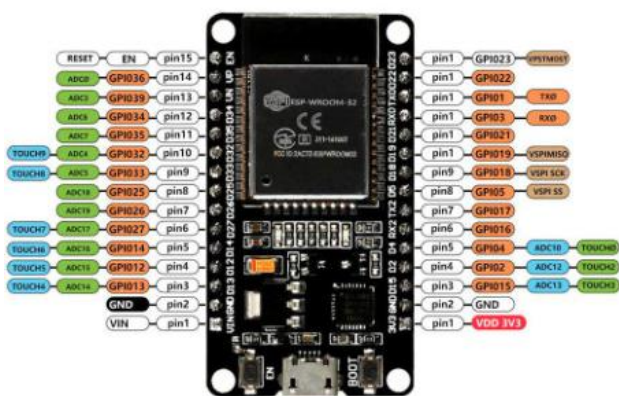
2.2. Vi điều khiển và các linh kiện được sử dụng trong mô hình bãi giữ xe thông minh [7]

2.2.1. Vi điều khiển ESP32 và Arduino Uno R3

❖ Giới thiệu về ESP32

ESP32 là vi điều khiển lõi kép 32-bit do Espressif phát triển, rất phổ biến trong các dự án IoT nhờ:

- Tích hợp Wi-Fi, Bluetooth.
- Hỗ trợ ADC, UART, I2C, PWM, SPI...
- Hoạt động ở điện áp 3.3V.
- Nhiều chân GPIO để kết nối thiết bị ngoại vi.
- Giá thành rẻ, dễ lập trình bằng Arduino IDE



Hình 2.5: Vi điều khiển ESP32 32pin

Trong đề tài, ESP32 là trung tâm điều khiển, có nhiệm vụ:

- Trao đổi dữ liệu với vi điều khiển Arduino.
- Kết nối mạng và giao tiếp web/app.
- Giao tiếp với màn hình LCD để hiển thị (trạng thái chỗ trống, mức phí phải trả).
- Điều khiển động cơ servo barie.

❖ Giới thiệu về Arduino Uno R3

Tổng quan thông số của Arduino Uno R3

- Vi xử lý: ATmega328P
- Điện áp hoạt động: 5 Volts
- Điện áp vào giới hạn: 7 đến 20 Volts
- Dòng tiêu thụ: khoảng 30mA
- Số chân Digital I/O: 14 (với 6 chân là PWM)
- UART: 1
- I2C: 1
- SPI: 1
- Số chân Analog: 6
- Dòng tối đa trên mỗi chân I/O: 30 mA
- Dòng ra tối đa (5V): 500 mA

- Dòng ra tối đa (3.3V): 50 mA
- Bộ nhớ flash: 32 KB với 0.5KB dùng bởi bootloader
- SRAM: 2 KB
- EEPROM: 1 KB
- Clock Speed: 16 MHz



Hình 2.6: Arduino Uno R3

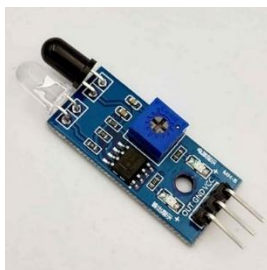
Trong đề tài, ARDUINO là trung tâm điều khiển, có nhiệm vụ:

- Nhận dữ liệu từ cảm biến (IR sensor – LM393), để phát hiện xe có đang ở vị trí đỗ.
- Đọc thẻ RFID, phục vụ xác thực xe/người dùng hợp lệ khi ra vào bãi.
- Xử lý logic và cấp tín hiệu điều khiển, dựa trên tín hiệu từ cảm biến và RFID mà gửi lệnh đóng/mở cổng barie(servo) cho xe vào/ra bãi.

2.2.2. Cảm biến hồng ngoại (LM393)

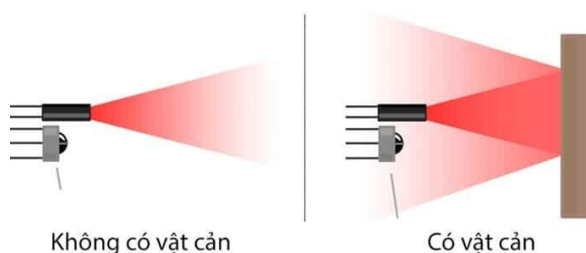
- Cấu tạo cảm biến hồng ngoại thường bao gồm các bộ phận:
 - Thiết bị thu hồng ngoại: giúp nhận tín hiệu hay phát hiện bức xạ hồng ngoại phản xạ trở lại.
 - Đèn led hồng ngoại: thiết bị phát ra nguồn sáng hồng ngoại (đối với cảm biến chủ động)
 - Điện trở: thiết bị có tác dụng cản trở cường độ dòng điện quá lớn chạy qua thiết bị, tránh làm hệ thống chập cháy.

- Dây dẫn: Có tác dụng kết nối các chi tiết tạo nên cảm biến hồng ngoại hoạt động ổn định.



Hình 2.7: Cảm biến hồng ngoại (IR sensor – LM393)

- Trong đề tài, IR sensor là khối cảm biến, có nhiệm vụ:
 - Phát hiện có xe tại vị trí đỗ.
 - Giúp hệ thống xác định số lượng chỗ đỗ còn trống.
- Nguyên lý hoạt động của cảm biến IR hoạt động bằng cách phát ra tia sáng hồng ngoại và nhận lại tia phản xạ từ vật thể phía trước. Nếu có vật (như xe) chắn trước cảm biến, tia sáng bị phản xạ trở lại và mắt thu của cảm biến sẽ nhận được tín hiệu đó. Khi đó, cảm biến gửi tín hiệu điện về cho Arduino để báo là có vật ở trước mặt.



Hình 2.8: Nguyên lý hoạt động của IR sensor

2.2.3. RFID – RC522

- ❖ Phân cứng của RFID gồm 3 phần chính: Thẻ RFID, Đầu đọc RFID và Ăng-ten RFID.
 - Thẻ RFID (RFID Tag): Thiết bị lưu trữ dữ liệu định danh, thường tích hợp trong thẻ từ, móc khóa hoặc điện thoại có NFC. Khi đưa thẻ lại gần khóa, hệ thống sẽ tự động nhận dạng và cấp quyền mở cửa. Có hai loại:
 - Thẻ thụ động: Không dùng pin, nhận năng lượng từ đầu đọc. Thường được dùng trong thẻ từ hoặc tag dán.
 - Thẻ chủ động: Có pin, tự phát tín hiệu. Dùng cho môi trường cần khoảng cách quét xa và bảo mật cao hơn.
 - Đầu đọc RFID (RFID Reader): Tích hợp bên trong khóa cửa thông minh, đảm nhiệm vai trò quét và xử lý tín hiệu từ thẻ RFID. Tùy vào loại thẻ sử dụng, đầu đọc có thể phát sóng chủ động hoặc chỉ nhận tín hiệu.

- Ăng-ten RFID: Được tích hợp sẵn trong thân khóa, giúp truyền – nhận dữ liệu qua sóng radio. Ăng-ten ảnh hưởng trực tiếp đến tốc độ nhận dạng, độ ổn định và phạm vi quét thẻ.



Hình 2.9: RFID – RC522

- ❖ Trong đề tài, RFID là khối cảm biến, có nhiệm vụ:
 - RFID cho phép mở barie và ghi nhận xe ra/vào bãi một cách tự động.
 - Nâng cao an ninh.

2.2.4. Camera ESP32 – CAM

- ❖ Thông số kỹ thuật:
 - Model: ESP32-CAM-MB
 - Sử dụng để nạp chương trình và giao tiếp truyền dữ liệu với máy tính cho mạch ESP32-CAM.
 - Điện áp sử dụng: 5VDC từ cổng MicroUSB
 - IC chuyển giao tiếp USB-UART: CH340
 - Tích hợp hai nút nhấn RST và IO0 sử dụng trong quá trình nạp chương trình.
 - Kích thước: 27x40mm



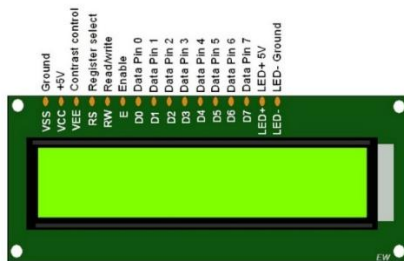
Hình 2.10:ESP32– CAM - MB

- ❖ Trong đề tài, ESP32 CAM là khối giám sát, có nhiệm vụ:
 - Chụp và truyền hình ảnh, hỗ trợ nhận diện phương tiện (biển số xe).
 - Phục vụ xác nhận phương tiện hợp lệ và giám sát an ninh hoặc đối chiếu khi xảy ra sự cố.
 - Tăng tính an toàn.

2.2.5. Các linh kiện khác

2.2.5.1. LCD (16x2)

- ❖ Thông số kỹ thuật LCD 16×2
 - LCD 16×2 được sử dụng để hiển thị trạng thái hoặc các thông số.
 - LCD 16×2 có 16 chân trong đó 8 chân dữ liệu (D0 – D7) và 3 chân điều khiển (RS, RW, EN).
 - 5 chân còn lại dùng để cấp nguồn và đèn nền cho LCD 16×2.
 - Các chân điều khiển giúp ta dễ dàng cấu hình LCD ở chế độ lệnh hoặc chế độ dữ liệu.
 - Chúng còn giúp ta cấu hình ở chế độ đọc hoặc ghi.
 - LCD 16×2 có thể sử dụng ở chế độ 4 bit hoặc 8 bit tùy theo ứng dụng ta đang làm.



Hình 2.11: Màn hình LCD 16x2

- ❖ Trong đề tài, LCD là khối hiển thị, có nhiệm vụ:
 - Hiển thị số lượng chỗ còn trống
 - Cung cấp thông tin mức phí giữ xe mà khách hàng phải thanh toán.

2.2.5.2 .Micro servo 9g -SG90

- ❖ Thông số kỹ thuật
 - Điện áp hoạt động: 4.8-5VDC
 - Tốc độ: 0.12 sec/ 60 deg (4.8VDC)
 - Momen xoắn: 1.8kg/cm
 - Kích thước: 22.2x11.8.32 mm
 - Trọng lượng: 9g.
 - Nhiệt độ hoạt động: 0 °C – 55 °C
- ❖ Phương pháp điều khiển PWM:
 - Độ rộng xung 0.5ms ~ 2.5ms tương ứng 0-180 độ
 - Tần số 50Hz, chu kỳ 20ms
- ❖ Sơ đồ dây:
 - Đỏ: Dương nguồn
 - Nâu: Âm nguồn
 - Cam: Tín hiệu



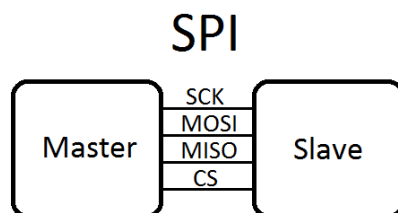
Hình 2.12: Micro servo 9g -SG90

- ❖ Trong đề tài, Servo SG90 là khối vận hành, có nhiệm vụ:
 - Đóng/mở barie tự động.
 - Tối ưu hóa hiệu quả quản lý ra/vào và phòng tránh tình trạng ùn tắc tại cổng.

2.3. Các chuẩn giao tiếp và giao thức truyền thông được sử dụng trong mô hình bãi giữ xe thông minh

2.3.1. Chuẩn giao tiếp SPI [8]

SPI (Serial Peripheral Interface) là một giao diện truyền thông đồng bộ được sử dụng để giao tiếp giữa các thiết bị trong hệ thống điện tử. Nó cho phép truyền dữ liệu giữa một thiết bị master và nhiều thiết bị slave thông qua các đường tín hiệu chung.



Hình 2.13: Chuẩn giao tiếp SPI

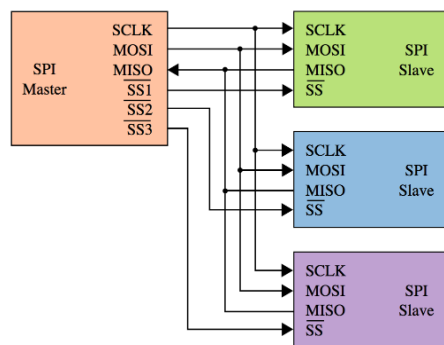
❖ Cấu tạo của giao tiếp SPI:

- SCLK (Serial Clock): Tín hiệu xung đồng hồ điều khiển cho việc đồng bộ truyền dữ liệu.
- MOSI (Master Out Slave In): Tín hiệu dữ liệu được truyền từ thiết bị master tới các thiết bị slave.
- MISO (Master In Slave Out): Tín hiệu dữ liệu được truyền từ các thiết bị slave về thiết bị master.
- SS/CS (Slave Select/Chip Select): Tín hiệu được sử dụng để chọn thiết bị slave nào sẽ tham gia truyền thông.

Trong quá trình truyền thông, thiết bị master điều khiển việc gửi và nhận dữ liệu bằng cách đồng bộ với các thiết bị slave thông qua tín hiệu SCLK. Thiết bị master chọn thiết bị slave cần truyền dữ liệu bằng cách đưa tín hiệu SS/CS về mức thấp.

Giao tiếp SPI được sử dụng rộng rãi trong các ứng dụng như viễn thông, điều khiển ngoại vi, giao tiếp với các cảm biến, mạch điều khiển đèn LED và nhiều thiết bị điện tử khác.

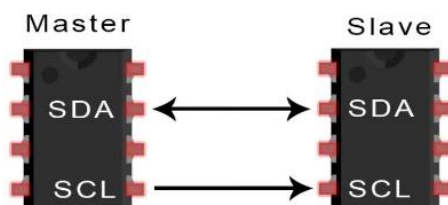
- ❖ **Nguyên lý hoạt động:** dựa trên việc truyền dữ liệu tuần tự giữa một thiết bị master và một hoặc nhiều thiết bị slave. Giao tiếp SPI sử dụng một tín hiệu clock chung và các tín hiệu truyền dữ liệu đồng thời trên các kênh tương ứng.



Hình 2.14: Sơ đồ kết nối giao tiếp SPI giữa một thiết bị Master (vi điều khiển) và nhiều thiết bị Slave (cảm biến, các linh kiện khác).

2.3.2. Chuẩn giao tiếp I2C [9]

Giao thức I2C (Inter-Integrated Circuit) là một giao thức truyền thông nối tiếp được phát triển bởi Philips (hiện tại là NXP Semiconductors) vào những năm 1980. Nó cho phép truyền dữ liệu giữa các thiết bị điện tử như vi điều khiển, cảm biến,... thông qua hai dây SDA và SCL.



Hình 2.15: Sơ đồ kết nối giao tiếp I2C giữa một thiết bị Master và nhiều thiết bị Slave

Chuẩn giao tiếp I2C được xây dựng trên nguyên tắc Master-Slave, trong đó một Master điều khiển quá trình truyền và nhận dữ liệu từ các Slave. Một Master có thể kết nối với nhiều Slave, và mỗi Slave được xác định bằng một địa chỉ duy nhất.

SDA và SCL của giao thức I2C sử dụng cơ chế truyền thông đồng bộ, trong đó tín hiệu (SCL) được Master điều khiển. Dữ liệu (bit) được truyền từng bit một qua đường SDA.

❖ Nguyên lý hoạt động:

- Thiết bị Master tạo điều kiện bắt đầu bằng cách chuyển đường dữ liệu (SDA) từ mức điện áp cao xuống mức điện áp thấp trong khi đường SCL đang ở mức điện áp cao.
- Sau khi bắt đầu, thiết bị Master gửi địa chỉ của thiết bị Slave mà nó muốn truyền dữ liệu đến. Địa chỉ này được gửi dưới dạng 7 hoặc 10 bit và đi kèm với một bit đọc/ghi (R/W) để xác định hoạt động là ghi (0) hay đọc (1).

- Sau khi gửi địa chỉ, thiết bị nhận (Slave) kiểm tra địa chỉ được gửi và gửi lại một bit ACK (mức điện áp thấp) nếu địa chỉ khớp. Nếu địa chỉ không khớp, thiết bị nhận sẽ không gửi bất kỳ gì và đường dữ liệu (SDA) sẽ vẫn ở mức điện áp cao.
- Master gửi hoặc nhận khung dữ liệu.
- Sau khi xác nhận địa chỉ, thiết bị Master có thể truyền hoặc nhận dữ liệu từ thiết bị nhận. Dữ liệu được truyền theo từng byte (8 bit) và sau mỗi byte, thiết bị nhận sẽ gửi một bit ACK (mức điện áp thấp) nếu dữ liệu được nhận thành công.
- Khi quá trình truyền dữ liệu hoàn thành, thiết bị chủ tạo điều kiện dừng bằng cách chuyển đường dữ liệu (SDA) từ mức điện áp thấp lên mức điện áp cao trong khi đường SCL đang ở mức điện áp cao.

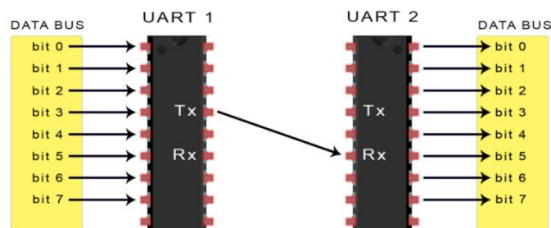
2.3.3. Chuẩn giao tiếp UART [10]

UART (Universal Asynchronous Receiver-Transmitter – Bộ truyền nhận dữ liệu không đồng bộ) là một giao thức truyền thông phần cứng dùng giao tiếp nối tiếp không đồng bộ và có thể cấu hình được tốc độ.

Giao thức UART là một giao thức đơn giản và phổ biến, bao gồm hai đường truyền dữ liệu độc lập là TX (truyền) và RX (nhận). Dữ liệu được truyền và nhận qua các đường truyền này dưới dạng các khung dữ liệu (data frame) có cấu trúc chuẩn, với một bit bắt đầu (start bit), một số bit dữ liệu (data bits), một bit kiểm tra chẵn lẻ (parity bit) và một hoặc nhiều bit dừng (stop bit).

Thông thường, tốc độ truyền của UART được đặt ở một số chuẩn, chẳng hạn như 9600, 19200, 38400, 57600, 115200 baud và các tốc độ khác. Tốc độ truyền này định nghĩa số lượng bit được truyền qua mỗi giây. Các tốc độ truyền khác nhau thường được sử dụng tùy thuộc vào ứng dụng và hệ thống sử dụng.

❖ Nguyên lý hoạt động:



Hình 2.16: Nguyên lý hoạt động của chuẩn giao tiếp UART

- UART truyền dữ liệu nối tiếp, theo 1 trong 3 chế độ:
 - Simplex: Chỉ tiến hành giao tiếp một chiều
 - Half duplex: Dữ liệu sẽ đi theo một hướng tại 1 thời điểm.
 - Full duplex: Thực hiện giao tiếp đồng thời đến và đi từ mỗi master và slave.

- Chân Tx (truyền) của một chip sẽ kết nối trực tiếp với chân Rx (nhận) của chip khác và ngược lại. Quá trình truyền dữ liệu thường sẽ diễn ra ở 3.3V hoặc 5V. Uart là một giao thức giao tiếp giữa một master và một slave. Trong đó 1 thiết bị được thiết lập để tiến hành giao tiếp với chỉ duy nhất 1 thiết bị khác.
- Dữ liệu truyền đến và đi từ Uart song song với thiết bị điều khiển. Khi tín hiệu gửi trên chân Tx (truyền), bộ giao tiếp Uart đầu tiên sẽ dịch thông tin song song này thành dạng nối tiếp và sau đó truyền tới thiết bị nhận. Chân Rx (nhận) của Uart thứ 2 sẽ biến đổi nó trở lại thành dạng song song để giao tiếp với các thiết bị điều khiển.
- Dữ liệu truyền qua Uart sẽ đóng thành các gói (packet). Mỗi gói dữ liệu chứa 1 bit bắt đầu, 5 – 9 bit dữ liệu (tùy thuộc vào bộ Uart), 1 bit chẵn lẻ tùy chọn và 1 bit hoặc 2 bit dừng.
- Quá trình truyền dữ liệu Uart sẽ diễn ra dưới dạng các gói dữ liệu này, bắt đầu bằng 1 bit bắt đầu, đường mức cao được kéo dài xuống thấp. Sau bit bắt đầu là 5 – 9 bit dữ liệu truyền trong khung dữ liệu của gói, theo sau là bit chẵn lẻ tùy chọn để nhằm xác minh việc truyền dữ liệu thích hợp. Sau cùng, 1 hoặc nhiều bit dừng sẽ được truyền ở nơi đường đặt tại mức cao. Vậy là sẽ kết thúc việc truyền đi một gói dữ liệu.

2.3.4. Giao thức WebSocket [11]

WebSocket là một giao thức truyền thông cung cấp các kênh liên lạc song công hoàn toàn qua một kết nối TCP duy nhất giữa máy khách và máy chủ. Không giống như HTTP truyền thống tuân theo mô hình phản hồi yêu cầu, giao thức này cho phép giao tiếp hai chiều. Điều này có nghĩa là máy khách và máy chủ có thể gửi dữ liệu cho nhau bất cứ lúc nào, giúp dữ liệu được truyền đi nhanh chóng mà không cần phải tải lại trang web.

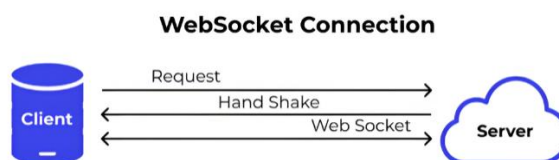
❖ Nguyên lý hoạt động:

Về bản chất, WebSocket có tính chất hai chiều, nghĩa là giao tiếp diễn ra qua lại giữa máy khách-máy chủ. Do đó, quá trình hoạt động của giao thức này vẫn cần bắt đầu bằng một "bắt tay" (handshake) đặc biệt, sau đó là duy trì một kết nối liên tục, cho phép truyền dữ liệu một cách nhanh chóng và hiệu quả. Toàn bộ quá trình hoạt động của WebSocket sẽ diễn ra như sau:

- Đầu tiên, trình duyệt gửi một yêu cầu HTTP đặc biệt đến máy chủ, yêu cầu mở một kết nối WebSocket. Yêu cầu này bao gồm một tiêu đề (Upgrade header) HTTP/1.1 thông báo cho máy chủ biết rằng trình duyệt muốn "nâng cấp" kết nối từ HTTP sang WebSocket.
- Nếu máy chủ có hỗ trợ giao thức WebSocket, nó sẽ phản hồi với một tiêu đề HTTP để xác nhận việc chuyển đổi. Khi tiêu đề này được trình duyệt tiếp nhận, kết nối WebSocket sẽ chính thức được thiết lập.
- Sau khi kết nối được thiết lập, máy chủ và trình duyệt đã có thể gửi dữ liệu lẫn nhau mà không cần yêu cầu hoặc phản hồi. Điều này giúp giảm thiểu độ trễ và tăng tốc độ truyền dữ liệu. Dữ liệu được truyền qua WebSocket được đóng gói trong các "khung" (Ws

frame). Mỗi khung có thể chứa dữ liệu văn bản hoặc nhị phân, trong đó mỗi bên lại có thể gửi và nhận dữ liệu độc lập với nhau.

- Khi có bất kỳ bên nào chủ động đóng kết nối, giao thức WebSocket sẽ được đóng lại và quá trình truyền tin sẽ kết thúc.



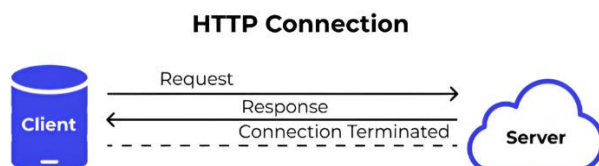
Hình 2.17: Nguyên lý hoạt động của giao thức WebSocket

2.3.5. Giao thức HTTP [12]

HTTP (HyperText Transfer Protocol), là một giao thức truyền tải siêu văn bản, giúp cho các máy tính có thể giao tiếp với nhau qua mạng. HTTP là nền tảng của World Wide Web kết nối giữa máy chủ (server) và máy khách (client) trong cùng một hệ thống mạng. Điều này cho phép chúng ta truy cập vào các trang web, tải xuống các tệp tin, xem các hình ảnh, video... Ban đầu khi được thiết kế vào những năm 90, HTTP là một giao thức linh hoạt có khả năng mở rộng theo thời gian. Giao thức này hoạt động trên nền tảng TCP/IP và thường được truyền qua kết nối mã hóa TLS để bảo vệ dữ liệu. Mặc dù về lý thuyết, bất kỳ giao thức truyền tải đáng tin cậy nào cũng có thể được áp dụng.

❖ Nguyên lý hoạt động:

- Giao thức HTTP (HyperText Transfer Protocol) là cách mà thiết bị và máy chủ trao đổi dữ liệu với nhau qua mạng. HTTP hoạt động theo mô hình client – server, nghĩa là thiết bị người dùng (client) gửi yêu cầu lên máy chủ (server), sau đó máy chủ xử lý và trả về kết quả. Ví dụ, khi bạn mở trang web hoặc bấm nút “đặt chỗ”, trình duyệt sẽ gửi yêu cầu HTTP lên server để lấy hoặc gửi thông tin, rồi server phản hồi lại dữ liệu cần thiết.
- Trong bãi giữ xe thông minh, HTTP được dùng để trao đổi dữ liệu không yêu cầu thời gian thực. Ví dụ: tải danh sách chỗ trống, gửi yêu cầu đặt chỗ, đăng nhập, hoặc xem lịch sử gửi xe. Mỗi yêu cầu chỉ diễn ra khi người dùng thực hiện hành động, giúp hệ thống hoạt động đơn giản, dễ quản lý và phù hợp với ứng dụng web.



Hình 2.18: Nguyên lý hoạt động của giao thức HTTP

CHƯƠNG 3: THIẾT KẾ MÔ HÌNH BÃI GIỮ XE THÔNG MINH

3.1. Yêu cầu đối với mô hình bãi giữ xe thông minh được thiết kế

❖ **Về phần cứng:**

- Hệ thống sử dụng cảm biến đặt ở vị trí giữ xe để xác định trạng thái giữ xe (trống hoặc có xe). Sau đó, gửi tín hiệu về bộ điều khiển trung tâm.
- Hệ thống barie (rào chắn tự động) cho phép kiểm soát ra vào bãi chứa xe bằng cách điều khiển đóng/mở tự động khi khách hàng đọc đúng mã đã được cấp trước đó.
- Màn hình LCD hiển thị thông tin tính tiền và trạng thái bãi đỗ.
- Các thiết bị hỗ trợ khác như điều khiển máy tính, ESP32 CAM để nhận diện biển số xe, thẻ đọc đầu, kết nối mạng LAN hoặc Internet để đồng bộ thông tin.

❖ **Về phần mềm:**

- Phần mềm quản lý bãi đậu xe có thể hoạt động trên web, cung cấp giao diện để khách hàng đặt chỗ trước, nhập thông tin đăng ký như tên, biển số xe, thời gian dự kiến đến, số điện thoại.
- Hệ thống ghi nhận cơ sở dữ liệu, thông tin khách hàng được quản lý.
- Tự động cấp mã cho khách hàng sử dụng với mục đích giữ chỗ, khi đến bãi khách hàng đọc mã đã được cấp để xác thực, camera sẽ chụp, lưu lại biển số xe, nhận thẻ RFID để mở rào chắn khi ra/vào bãi giữ xe.
- Theo dõi thời gian gửi xe và tính tiền dựa trên thời gian thực tế trong bãi.
- Đồng bộ trạng thái giữ xe theo thời gian thực trên web để khách hàng dễ dàng xem còn chỗ trống tại từng vị trí đỗ hay không.

❖ **Về khả năng của hệ thống:**

- Khách hàng có thể đặt trước chỗ giữ xe trên web khoảng 30 phút trước khi đến.
- Tại thời điểm đăng ký hệ thống sẽ kiểm tra và hiển thị vị trí nào còn trống dựa vào cảm biến.
- Tạo ra mã cho khách hàng, mã số này có thể là một chuỗi số được bảo mật và lưu trên cơ sở dữ liệu của hệ thống.
- Giảm số lượng chỗ trống khi khách hàng đặt chỗ và cập nhật trạng thái giữ xe theo thời gian thực.
- Tự động tính tiền gửi xe khi khách hàng lấy xe ra, hiển thị số tiền trên LCD.
- Tối ưu hóa công việc quản lý bãi đỗ, giảm thiểu thời gian khách tìm chỗ đỗ và giảm nhân lực quản lý.

❖ **Phạm vi sử dụng:**

- Hệ thống phù hợp cho bãi giữ xe ô tô tại các trung tâm thương mại, khu chung cư, bệnh viện, văn phòng, trường học,...hay bãi giữ xe công cộng.
- Hệ thống hiện tại thiết kế với quy mô nhỏ. Thực tế có thể mở rộng số lượng vị trí đỗ và người dùng.

❖ **Giới hạn khi triển khai hệ thống:**

- Yêu cầu phải có mạng ổn định (wifi) để đồng bộ hóa thời gian dữ liệu.

- Phải đảm bảo an toàn thông tin khách hàng, bảo mật mã để tránh giả mạo hoặc sử dụng sai mục tiêu.
- Có thể phát hiện ra chỗ trống nhưng chưa thể xác định và đưa ra chỉ dẫn về vị trí chính xác.

3.1.2. Tính năng của các thành phần trong hệ thống

- **Đặt chỗ trước qua web:** Khách hàng có thể tìm hiểu, đăng ký và đặt trước vị trí giữ xe qua trang web. Kiểm tra hệ thống và xác thực các vị trí còn trống, cấp độ xác thực mã hóa nhận được trong thời gian lưu trữ (ví dụ: 30 phút). Người dùng đã nhận được mã và hệ thống tự động ghi nhận thời gian và vị trí đã đặt. Khi đến bãi, chỉ cần xuất trình mã để nhận thẻ RFID ra/vào bãi giữ xe.
- **Kiểm soát quá trình tự động vào/ra bãi đỗ bằng thẻ RFID:** Hệ thống sử dụng RFID để nhận dạng phương tiện hoặc người dùng định nghĩa cho mỗi lượt truy cập. Bộ điều khiển trung tâm xác thực và tự động kích hoạt barie/servo để mở cổng khi hợp lệ, giảm thiểu tối đa nhân công thủ công và nâng cao bảo mật.
- **Tích hợp camera thông minh (ESP32-CAM):** ESP32-CAM giúp ghi lại hình ảnh, hỗ trợ nhận dạng biển số xe, dịch vụ minh chứng, giám sát an ninh cũng xác thực hợp lệ phương tiện. Hình ảnh và dữ liệu được truyền về máy chủ để quản lý, đối chiếu khi cần thiết.
- **Hệ thống cảm biến tạo ra khoảng trống theo thời gian thực:** Các cảm biến hồng ngoại xác định chính xác tình trạng từng vị trí giữ xe (còn trống/có xe), cập nhật liên tục dữ liệu lên web cũng như màn hình LCD tại bãi để người dùng và quản lý dễ dàng theo dõi.
- **Hiển thị thông tin trên LCD và web:** LCD 16x2 hiển thị số lượng chỗ trống còn lại, mức phí dự kiến. Giao diện web đồng bộ trạng thái theo thời gian thực giúp khách hàng giám sát trực tuyến và quản lý thuận tiện hơn.
- **Thanh toán và tính phí tự động:** Khi xe ngưng đỗ, hệ thống tự động tính toán thời gian gửi xe và sau đó hiển thị cho khách hàng trên LCD cũng như cập nhật thông tin lên web, sẵn sàng cho quá trình thanh toán và tra cứu lịch sử gửi xe.
- **Cảnh báo, bảo mật và quản lý linh hoạt:** Hệ thống hỗ trợ chức năng cảnh báo khi đăng ký/đăng nhập không thành công, các vị trí đầy/chưa đặt chỗ hợp lệ, kiểm soát chặt chẽ từng lần vào ra, đảm bảo an toàn tài sản và thông tin khách hàng, đáp ứng nhu cầu quản lý linh hoạt cho bãi xe nhiều quy mô.

3.1.3. Các trường hợp sử dụng

- ❖ **Trường hợp 1:** Khách hàng muốn tìm chỗ giữ xe

Bảng 3.1: Trường hợp ứng dụng thực tế (user case 1)

Các yếu tố	Định nghĩa
Kích hoạt	Mở web, xem đã đăng ký chưa:

	- Nếu chưa, KH đăng ký trên web đúng quy định (ghi tên KH, biển số xe, mail, số điện thoại,...) sau đó đăng nhập. - Nếu rồi, đăng nhập.
Điều kiện tiên quyết	KH xem thông tin về bãi giữ xe. Còn chỗ trống tại thời điểm đăng ký hay không. Xác nhận đặt chỗ.
Luồng cơ bản	Hệ thống cấp mã xác thực, ghi nhận thời gian được giữ chỗ của KH (30 phút). Giảm số lượng chỗ trống. Khi đến bãi xe, KH đọc mã đã được cấp. Nếu đúng mã và còn hiệu lực 30 phút, lưu biển số xe vào cơ sở dữ liệu, cấp thẻ RFID, mở cổng, cho xe vào và bắt đầu thời gian tính tiền. Khi KH lấy xe ra, so sánh với biển số xe đã lưu, quét thẻ RFID, chờ 5s, tính phí giữ xe, KH thanh toán và rời bãi.
Luồng thay thế	Khi đến bãi, nếu đọc mã sai. Không nhận KH/tính là khách vắng lai (Trường hợp 2)

❖ **Trường hợp 2:** Khách hàng đến trực tiếp bãi mà chưa đặt chỗ trước (Khách vắng lai).

Bảng 3.2: Trường hợp ứng dụng thực tế (user case 2)

Các yếu tố	Định nghĩa
Kích hoạt	Khách hàng đến bãi giữ xe.
Điều kiện tiên quyết	Bãi giữ xe còn chỗ trống.
Luồng cơ bản	Camera chụp và lưu biển số xe, cấp thẻ RFID, mở cổng cho xe vào và bắt đầu tính thời gian đỗ. Khi lấy xe ra, so sánh biển số xe đã lưu, quét thẻ, tính phí giữ xe. Khách hàng thanh toán và rời bãi.
Luồng thay thế	Nếu không còn chỗ trống, từ chối cho xe vào.

❖ **Trường hợp 3:** Khách hàng đã đăng ký tài khoản và đặt chỗ.

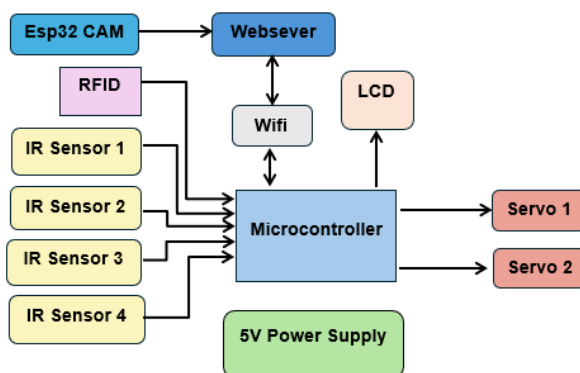
Bảng 3.3: Trường hợp ứng dụng thực tế (user case 3)

Các yếu tố	Định nghĩa
Kích hoạt	Khách hàng đến bãi giữ xe.
Điều kiện tiên quyết	Trước 30 phút.
Luồng cơ bản	Xác thực mã khách hàng. Camera chụp và lưu lại biển số xe, cấp thẻ RFID, mở cổng cho xe vào và bắt đầu tính thời gian đỗ. Khi lấy xe ra, so sánh với biển số xe đã lưu, quét thẻ, tính phí giữ xe. Khách hàng thanh toán và rời bãi.
Luồng thay thế	Nếu đến trễ sau 30 phút, tính là khách vắng lai (trường hợp 2)

3.2. Thiết kế phần cứng

3.2.1. Sơ đồ khối của hệ thống

❖ *Sơ đồ khối tổng quát:*



Hình 3.1: Sơ đồ khối tổng quát của hệ thống.

➤ *Thành phần chính và chức năng:*

- **Esp32 CAM:** Đóng vai trò chụp hình hoặc nhận diện phương tiện, truyền dữ liệu về Webserver để xử lý và lưu trữ.
- **Webserver:** Nơi lưu trữ, hiển thị và quản lý thông tin xe ra/vào, trạng thái bãi đỗ hoặc trạng thái barrier. Webserver giao tiếp với hệ thống qua mạng Wifi.
- **RFID:** Đầu đọc thẻ dùng để nhận diện người dùng/xe ra vào. Tín hiệu từ RFID được gửi đến vi điều khiển để xác thực.
- **4 cảm biến IR:** Các cảm biến hồng ngoại (IR Sensor 1, 2, 3, 4) lắp ở các vị trí trong bãi xe hoặc tại cổng ra/vào giúp nhận biết có xe tại vị trí quét. Tín hiệu cảm biến truyền vào vi điều khiển qua các chân digital.
- **Microcontroller:** Bộ não của hệ thống, nhận các tín hiệu từ cảm biến, RFID, ESP32 CAM rồi xử lý và điều khiển các động cơ servo. Ngoài ra, microcontroller

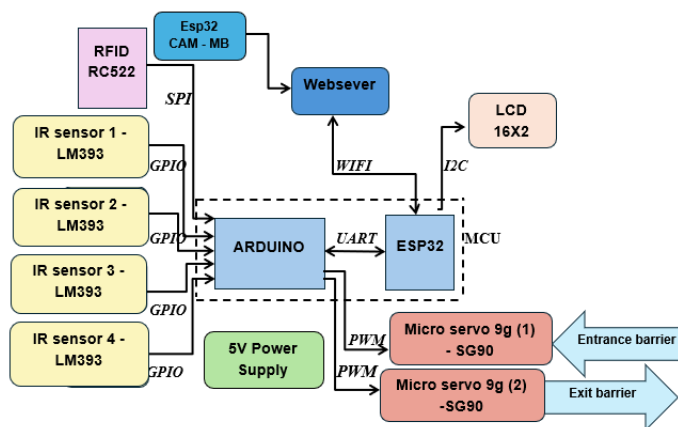
có thể truyền thông với LCD để cập nhật thông tin cho người dùng và giao tiếp với Webserver *qua Wifi*.

- **LCD:** Màn hình hiển thị trạng thái bãi xe, số lượng xe đang có hoặc thông báo dành cho người dùng.
- **Servo 1 & Servo 2:** Hai động cơ servo được dùng để đóng/mở barrier tại cổng vào và ra, giúp kiểm soát luồng xe.
- **5V Power Supply:** Cấp nguồn điện cho toàn bộ hệ thống, đảm bảo các thiết bị đều hoạt động liên tục.

➤ **Nguyên lý hoạt động tổng quát**

- Xe đến bãi đỗ, quét thẻ RFID; vi điều khiển kiểm tra thẻ.
- Nếu hợp lệ, vi điều khiển điều khiển Servo mở barrier cho xe vào/ra.
- Tín hiệu cảm biến xác nhận có xe, đồng thời cập nhật trạng thái lên Website và LCD.
- Số lượng chỗ trống, chỗ đã có xe đỗ, trạng thái barrier được cập nhật về Webserver và hiển thị ra LCD.
- Khi xe rời bãi cũng theo trình tự hoạt động như trên.

❖ **Sơ đồ khối chi tiết:**



Hình 3.2: Sơ đồ khối chi tiết của hệ thống.

➤ **Chức năng các khối**

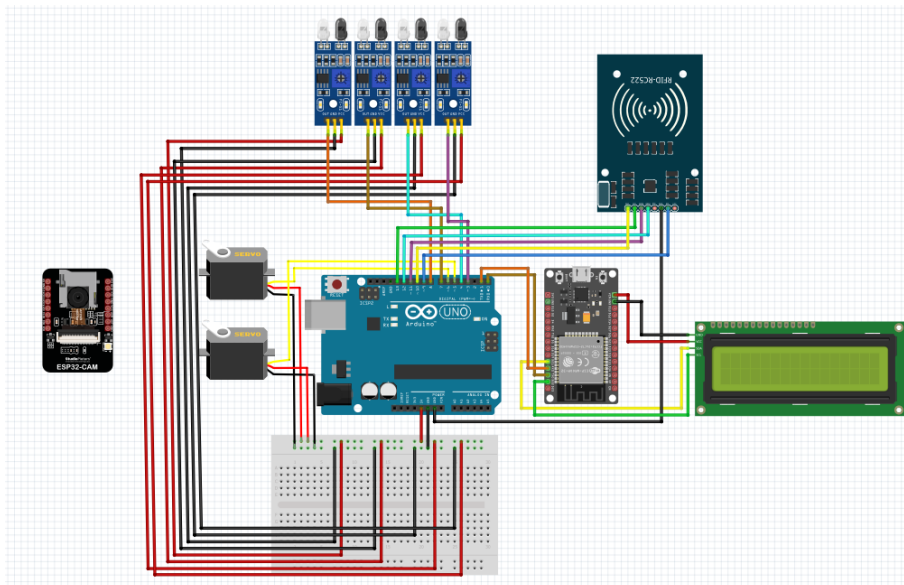
- **RFID RC522:** Đầu đọc thẻ RFID giúp nhận diện xe hoặc chủ xe tại cổng vào/ra, kết nối với Arduino qua giao thức SPI.
- **Esp32 CAM - MB:** Module camera giúp chụp hình biển số, truyền dữ liệu lên Webserver để quản lý hoặc lưu trữ. Đồng thời, Esp32 hỗ trợ gửi dữ liệu về website thông qua wifi.

- **IR sensor 1-4 (LM393):** Mỗi cảm biến hồng ngoại đặt tại các vị trí kiểm soát chỗ đậu xe, truyền tín hiệu trạng thái (phát hiện có xe hay không) về Arduino qua các chân GPIO.
- **ARDUINO:** Vi điều khiển trung tâm, nhận dữ liệu từ RFID và IR sensor, gửi lệnh điều khiển động cơ servo đóng/mở barrier, đồng thời giao tiếp với ESP32 qua UART.
- **ESP32:** Vi xử lý phụ, giao tiếp với Arduino qua UART. ESP32 kết nối wifi để đồng bộ dữ liệu với webserver và truyền thông tin hiển thị sang LCD qua I2C.
- **Micro servo 9g (SG90):** Hai động cơ servo SG90 dùng để điều khiển barrier cổng vào (entrance barrier) và ra (exit barrier), nhận lệnh từ Arduino bằng tín hiệu PWM.
- **LCD 16x2:** Màn hình hiển thị trạng thái bãi đỗ, số lượng xe, mức phí phải trả cho người sử dụng, nhận dữ liệu từ ESP32 qua I2C.
- **5V Power Supply:** Cấp nguồn cho toàn hệ thống, đảm bảo hoạt động ổn định cho các cảm biến, vi điều khiển và thiết bị ngoại vi.
- **Webserver:** Quản lý dữ liệu xe vào, ra, trạng thái bãi đỗ trên website thông qua ESP32 CAM.

➤ **Nguyên lý hoạt động**

- Khi xe đến cổng, quét thẻ RFID.
- Arduino xử lý và xác thực thông tin qua ESP32, kiểm tra trạng thái, gửi dữ liệu về webserver để cập nhật số xe ra/vào.
- Nếu hợp lệ, Arduino điều khiển servo để mở barrier cho xe di chuyển vào hoặc ra.
- Khi xe vào vị trí đỗ, cảm biến phát hiện có xe hay không.
- Dữ liệu số lượng xe đã đỗ trong bãi (hay số chỗ còn trống), trạng thái barrier được truyền qua ESP32 để hiển thị trên LCD, đồng thời đồng bộ lên website.

3.2.2. Sơ đồ nối các linh kiện



Hình 3.3: Sơ đồ nối các linh kiện

❖ **VCC: 5V, GND: GND**

- Cảm biến IR1 - OUT: 8 (ARDUINO)
- Cảm biến IR2 - OUT: 7 (ARDUINO)
- Cảm biến IR3 - OUT: 4 (ARDUINO)
- Cảm biến IR4 - OUT: 3 (ARDUINO)
- Servo 1 - OUT: 6 (ARDUINO)
- Servo 2 - OUT: 5 (ARDUINO)

❖ **VCC: 3.3V, GND: GND**

- RFID RC522:
 - RST: 9 (ARDUINO)
 - SDA: 10 (ARDUINO)
 - MOSI: 11 (ARDUINO)
 - MISO: 12 (ARDUINO)
 - SCK: 13 (ARDUINO)
 - GND: GND (ARDUINO)
 - VCC: Không nối

❖ **LCD (Nối với ESP32)**

➤ LCD I2C 16x2:

- VCC: VIN
- GND: GND
- SDA: D21
- SCL: D22

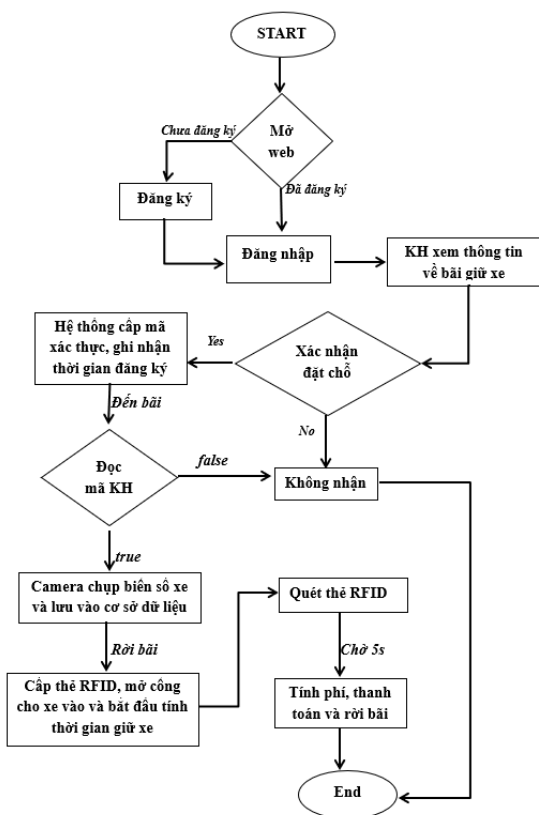
❖ **UART**

- RX0 (ESP32) -> TX0 (ARDUINO)
- TX0 (ESP32) -> RX0 (ARDUINO)

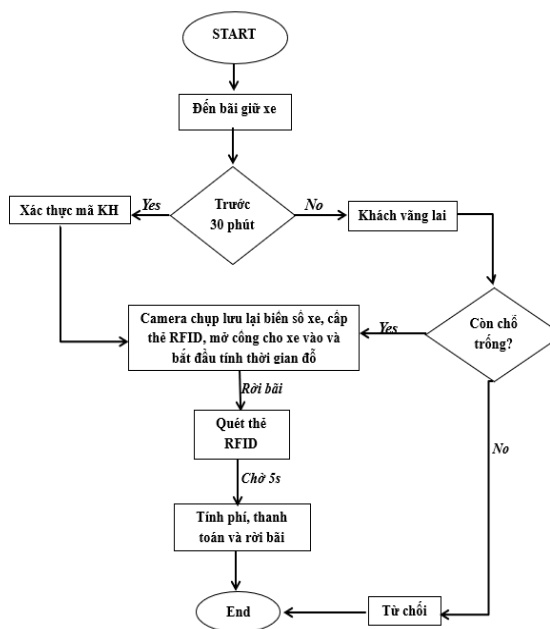
3.3. Thiết kế phần mềm

3.3.1. Lưu đồ hoạt động của hệ thống

*Lưu đồ 1: KH muốn tìm chỗ giữ xe



*Lưu đồ 2: Khách hàng đã đăng ký qua web



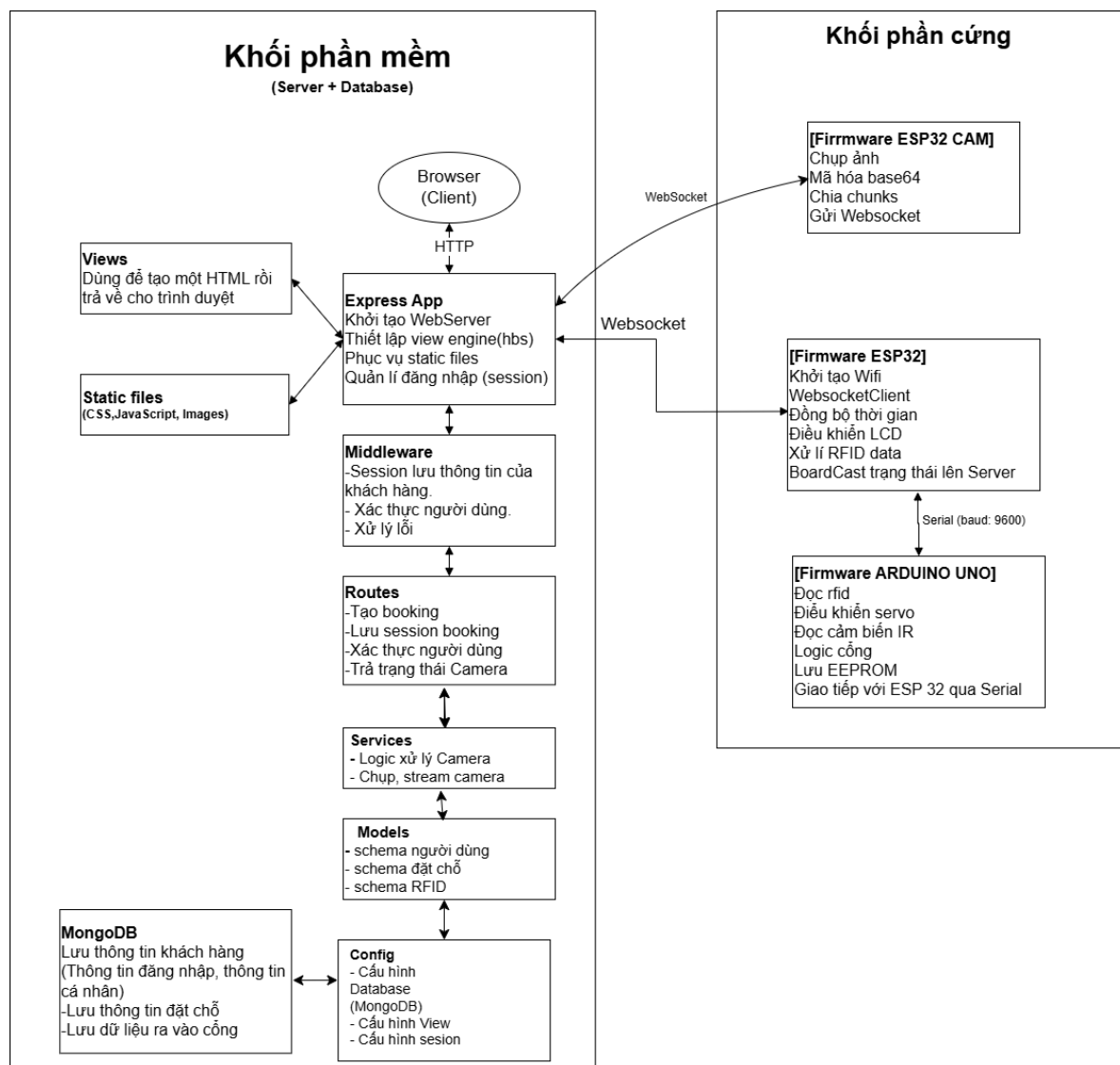
*Lưu đồ 3: Khách hàng đến trực tiếp bãi mà chưa đăng ký tài khoản và đặt chỗ (Khách vắng lai).



Hình 3.4: Lưu đồ hoạt động của hệ thống

3.3.2. Chương trình phần mềm

3.3.2.1. Cấu trúc chương trình



Hình 3.5: Sơ đồ khối phần mềm cho vi điều khiển

❖ Backend - Luồng xử lý từ Client đến Server

Browser (Client)

Người dùng (khách hàng muốn đỗ xe) truy cập website qua trình duyệt web thông qua giao thức HTTP để đặt chỗ đỗ xe.

Express App (Node.js Server)

Máy chủ web chính nhận yêu cầu từ Client với các chức năng:

- **Khởi tạo WebServer:** Tạo server để xử lý requests từ Client

- **Thiết lập view engine(bs):** Cấu hình Handlebars để render HTML động
- **Phục vụ static files:** Phục vụ các file tĩnh (CSS, JS, hình ảnh)
- **Quản lý đăng nhập (session):** Xác thực và duy trì phiên đăng nhập của khách hàng

Middleware

Lớp trung gian xác thực truy cập:

- **Session lưu thông tin của khách hàng:** Khi Client gửi yêu cầu đến Server, Middleware sẽ xác thực và lưu thông tin đăng nhập của khách hàng
- **Biết được khách hàng nào đang đăng nhập:** Middleware giúp Server nhận diện được người dùng hiện tại (để quản lý đặt chỗ của từng khách hàng)

Routes (Định tuyến)

Sau khi xác thực, khách hàng yêu cầu gì thông qua API từ Routes:

- **Tạo booking:** Xử lý đặt chỗ đỗ xe trước
- **Lưu session booking:** Lưu trạng thái đặt chỗ của khách hàng
- **Xác thực người dùng:** Kiểm tra quyền truy cập vào bãi đỗ
- **Trả trạng thái Camera:** Lấy hình ảnh từ camera giám sát bãi đỗ

Routes sẽ lấy dữ liệu từ MongoDB để xử lý (kiểm tra chỗ trống, thông tin đặt chỗ), sau đó trả ngược về phía Client.

MongoDB

Cơ sở dữ liệu lưu trữ toàn bộ thông tin hệ thống bãi đỗ xe:

- **Lưu thông tin khách hàng:** Thông tin đăng nhập, thông tin cá nhân, biển số xe
- **Thông tin đặt chỗ (booking):** Dữ liệu đặt chỗ trước, thời gian đặt, vị trí chỗ đỗ
- **Thông tin đăng nhập:** Tài khoản, mật khẩu
- **Dữ liệu ra vào cổng:** Lịch sử xe vào/ra bãi đỗ thông qua cảm biến và RFID

Static Files

Lưu trữ các file tĩnh cho website:

- **Hình ảnh:** Logo bãi đỗ, icon, sơ đồ bãi đỗ
- **CSS:** Thiết kế giao diện web đặt chỗ
- **JavaScript:** Các script phía client để tương tác

Views

Xuất file Handlebars hoàn chỉnh (HTML) để gửi về phía người dùng:

- **Hiển thị sơ đồ bãi đỗ** với các chỗ trống/đã đặt
- **Form đặt chỗ trước**
- **Lịch sử đỗ xe**
- **Gửi về Browser** để hiển thị

❖ Hardware - Các khối thiết bị IoT tại bãi đỗ xe

Khối 1: ESP32-CAM (Camera giám sát)

[Firmware ESP32 CAM] - Module camera giám sát bãi đỗ:

- **Chụp ảnh:** Chụp hình ảnh bãi đỗ xe, biển số xe
- **Mã hóa base64:** Chuyển đổi ảnh sang định dạng base64 để truyền qua mạng
- **Chia chunks:** Chia nhỏ dữ liệu ảnh thành các phần nhỏ (do ảnh có dung lượng lớn)
- **Gửi Websocket:** Truyền hình ảnh realtime lên Server để khách hàng xem trực tiếp

Khối 2: ESP32 (Bộ xử lý trung tâm)

[Firmware ESP32] - Vi điều khiển trung tâm điều khiển toàn bộ hệ thống:

- **Khởi tạo Wifi:** Kết nối mạng không dây với Server
- **WebsocketClient:** Nhận/gửi dữ liệu realtime với Server (trạng thái chỗ đỗ, lệnh mở cổng)
- **Đồng bộ thời gian:** Đồng bộ thời gian để ghi nhận giờ vào/ra
- **Điều khiển LCD:** Hiển thị thông tin (số chỗ trống, thông báo) trên màn hình tại cổng
- **Xử lý RFID data:** Xử lý dữ liệu từ thẻ RFID/thẻ xe của khách hàng
- **BoardCast trạng thái lên Server:** Gửi trạng thái bãi đỗ (số chỗ trống, xe vào/ra) lên Server qua Websocket

Khối 3: Arduino UNO (Điều khiển cổng và cảm biến)

[Firmware ARDUINO UNO] - Điều khiển phần cứng tại cổng vào/ra:

- **Đọc RFID:** Đọc thẻ xe/thẻ đặt chỗ của khách hàng (xác thực quyền vào bãi)

- **Điều khiển servo:** Mở/đóng cổng barrier (thanh chắn) tự động
- **Đọc cảm biến IR:** Phát hiện xe đang ở cổng (tránh đóng cổng khi xe chưa qua hết)
- **Logic cổng:** Xử lý logic mở/đóng cổng dựa trên RFID và cảm biến
- **Lưu EEPROM:** Lưu trữ thông tin thẻ RFID đã đăng ký vĩnh viễn
- **Giao tiếp với ESP32 qua Serial:** Truyền nhận dữ liệu với ESP32 qua UART

❖ **Luồng hoạt động tổng thể của Hệ thống Bãi Đỗ Xe Thông Minh:**

Luồng đặt chỗ trước (Web Application):

1. **Khách hàng** truy cập website → đăng nhập qua **Browser**
2. **Express App** nhận yêu cầu → **Middleware** xác thực khách hàng
3. Khách hàng chọn **đặt chỗ trước** → **Routes** xử lý
4. **Routes** kiểm tra chỗ trống trong **MongoDB** → tạo booking
5. **Views** render trang xác nhận + **Static files** → gửi về **Browser**
6. Khách hàng nhận mã QR/RFID để vào bãi đỗ

Luồng xe vào bãi đỗ (IoT):

1. Xe đến cổng → **Arduino UNO** đọc thẻ **RFID**
2. **Arduino** gửi dữ liệu qua Serial → **ESP32** xử lý
3. **ESP32** gửi yêu cầu xác thực lên **Server** qua Websocket
4. **Server** kiểm tra booking trong **MongoDB** → xác nhận hợp lệ
5. **ESP32** gửi lệnh → **Arduino** điều khiển **servo** → mở cổng
6. **Cảm biến IR** phát hiện xe qua → **Arduino** đóng cổng
7. **ESP32** cập nhật trạng thái (giảm số chỗ trống) → broadcast lên **Server**

Luồng giám sát realtime:

1. **ESP32-CAM** chụp ảnh bãi đỗ liên tục
2. Mã hóa base64 → chia chunks → gửi qua Websocket → **Server**
3. **Server** cập nhật **MongoDB** và hiển thị cho **Client**
4. Khách hàng có thể xem trực tiếp tình trạng bãi đỗ trên website

5. **ESP32** hiển thị số chỗ trống trên **LCD** tại cổng

Luồng xe ra khỏi bãi đỗ:

1. **Arduino** đọc RFID → gửi lên **ESP32**
2. **ESP32** xác thực → gửi lệnh mở cổng
3. **Arduino** mở servo → xe ra → IR phát hiện → đóng cổng
4. **ESP32** cập nhật (tăng số chỗ trống) → broadcast lên **Server**
5. **Server** lưu lịch sử ra vào trong **MongoDB**

➤ **Tóm lại:** Đây là hệ thống bãi đỗ xe thông minh toàn diện với:

- **Đặt chỗ trước** qua website
- **Tự động mở/đóng cổng** bằng RFID
- **Camera giám sát** realtime
- **Hiển thị số chỗ trống** trên LCD
- **Quản lý lịch sử** vào/ra
- **Đồng bộ dữ liệu** realtime giữa thiết bị IoT và Server Retry

3.3.2.2: Các thư viện được sử dụng trong firmware

❖ **esp32.ino:**

Bảng 3.4: Thư viện được sử dụng trong esp32.ino

Thư viện	Chức năng
#include <WiFi.h>	Kết nối WiFi
#include <WebSocketsClient.h>	Giao tiếp WebSocket
#include <NTPClient.h>	Đồng bộ thời gian
#include <WiFiUdp.h>	Giao thức UDP cho NTP

#include <ArduinoJson.h>	Xử lý chuỗi json
#include <Wire.h>	Giao tiếp I2C
#include <LiquidCrystal_I2C.h>	Hiển thị LCD

❖ **arduino.ino:**

Bảng 3.5: Thư viện được sử dụng trong arduino.ino

Thư viện	Chức năng
#include <Servo.h>	Điều khiển servo
#include <SPI.h>	Giao tiếp SPI
#include <MFRC522.h>	Đọc thẻ RFID
#include <EEPROM.h>	Lưu trữ dữ liệu
#include <ArduinoJson.h>	Xử lý chuỗi json

❖ **esp32_cam.ino:**

Bảng 3.6: Thư viện được sử dụng trong esp32_cam.ino

Thư viện	Chức năng
#include <WiFi.h>	Kết nối WiFi ESP32
#include <WebSocketsClient.h>	Giao tiếp WebSocket
#include <HttpClient.h>	Giao tiếp HTTP (dự phòng)
#include "esp_camera.h"	Driver camera ESP32
#include <ArduinoJson.h>	Xử lý dữ liệu JSON

<code>#include <Base64.h></code>	Mã hóa ảnh Base64
<code>#include "soc/soc.h"</code>	Cho phép truy cập thanh ghi hệ thống SoC
<code>#include "soc/rtc_cntl_reg.h"</code>	Cho phép ghi/đọc thanh ghi RTC

3.3.2.3. Các hàm quan trọng được sử dụng trong các firmware.

❖ **camService.js:**

Bảng 3.7: Chức năng chung của camService.js

Hàm	Chức năng chính
<code>recognizeLicensePlate()</code>	Nhận diện biển số xe từ ảnh
<code>saveImage()</code>	Lưu ảnh vào thư mục được chỉ định
<code>deleteOldImages()</code>	Dọn rác ảnh cũ
<code>comparePlates()</code>	So sánh hai biển số
<code>getImageInfo()</code>	Lấy thông tin file
<code>validateImage()</code>	Kiểm tra ảnh hợp lệ trước khi nhận diện

❖ **booking.js:**

Bảng 3.8: Chức năng chung của booking.js

Hàm	Chức năng chính
<code>const mongoose = require("mongoose");</code>	Nạp thư viện Mongoose để làm việc với MongoDB.
<code>const bookingSchema = new mongoose.Schema({...});</code>	Tạo khuôn mẫu dữ liệu (Schema) cho bảng “Booking”.

❖ **user.js:**

Bảng 3.9: Chức năng chung của user.js

Hàm	Chức năng chính
<code>const userSchema = new mongoose.Schema({...}, { timestamps: true });</code>	Tạo khuôn mẫu dữ liệu (Schema) cho người dùng.

❖ **confirm.js:***Bảng 3.10: Chức năng chung của confirm.js*

Hàm	Chức năng chính
<code>function confirmBooking() { ... }</code>	Hàm này sẽ chứa logic xác nhận đặt chỗ.
<code>document.addEventListener('DOMContentLoaded', function () { ... });</code>	Đợi toàn bộ HTML tải xong, rồi mới chạy đoạn mã bên trong.

❖ **manager.js:***Bảng 3.11: Chức năng chung của manager.js*

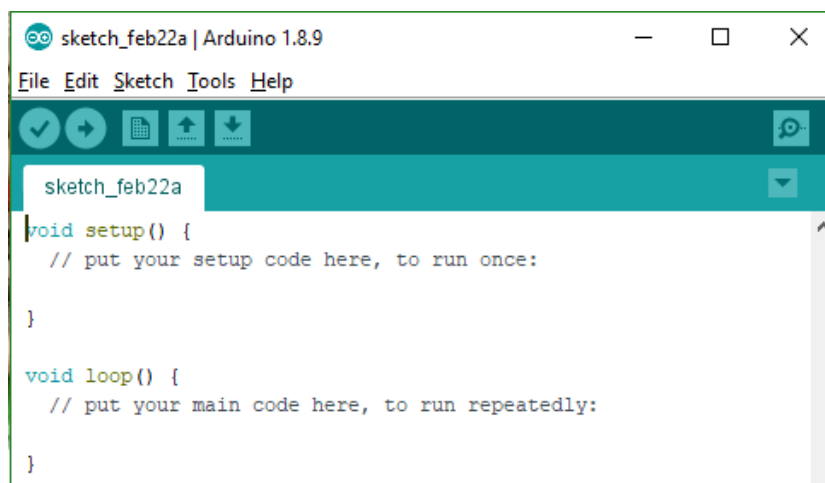
Hàm	Chức năng chính
<code>updateBookingStatus(bookingCode, newStatus)</code>	Cập nhật trạng thái đặt chỗ.
<code>formatVietnamTime(dateString)</code>	Định dạng thời gian theo múi giờ Việt Nam.
<code>initWebSocket()</code>	Kết nối WebSocket để cập nhật dữ liệu thời gian thực.
<code>autoRefreshImages()</code>	Tự động làm mới hình ảnh camera.
<code>updateCameraStatus(online)</code>	Hiển thị trạng thái camera (hoạt động / mất kết nối).
<code>addSecurityAlert(alert)</code>	Thêm cảnh báo an ninh lên giao diện.
<code>resolveAlert(button)</code>	Xử lý khi người quản lý tắt cảnh báo.

showNotification(message, type)	Hiển thị thông báo (info, warning, error).
setGateStatus(gateId, isOpen)	Cập nhật trạng thái cổng (mở / đóng).

3.3.3. Nhiệm vụ, chức năng và ý nghĩa của các phần mềm được sử dụng

❖ Arduino IDE

- **Nhiệm vụ:** Arduino IDE là môi trường lập trình mã nguồn mở dùng để viết, biên dịch và nạp chương trình cho các vi điều khiển Arduino và ESP32 trong hệ thống.
- **Chức năng:**
 - Cung cấp giao diện trực quan, dễ sử dụng, tích hợp công cụ kiểm tra và nạp chương trình.
 - Hỗ trợ quản lý thư viện và bảng mạch, giúp tương thích với nhiều loại cảm biến và mô-đun.
 - Hoạt động đa nền tảng (Windows, macOS, Linux) với cộng đồng hỗ trợ lớn.
- **Ý nghĩa:** Arduino IDE là công cụ cốt lõi giúp lập trình, kiểm thử và hiệu chỉnh bộ điều khiển trong hệ thống bãi giữ xe thông minh, đảm bảo tính ổn định và khả năng mở rộng của hệ thống.

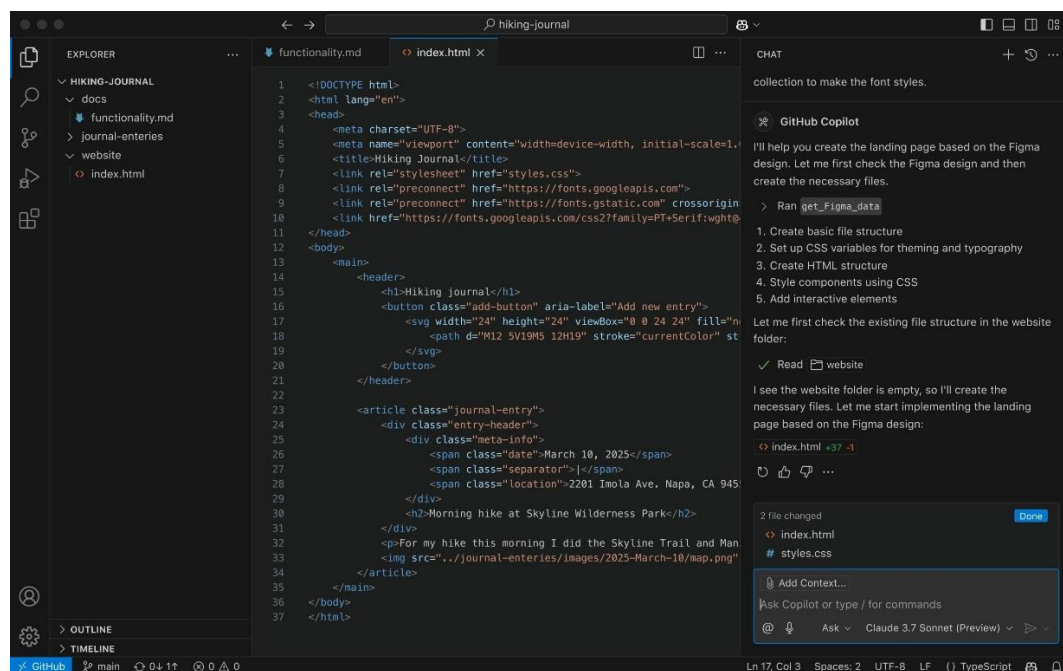


Hình 3.6: Giao diện phần mềm Arduino IDE

❖ Visual Studio Code (VS Code)

- **Nhiệm vụ:** Là trình soạn thảo mã nguồn chính phục vụ phát triển phần mềm điều khiển, kết nối thiết bị IoT và xây dựng giao diện web.
- **Chức năng:**
 - Hỗ trợ nhiều ngôn ngữ lập trình (C/C++, Python, JavaScript...).
 - Tích hợp công cụ gỡ lỗi, quản lý dự án và kiểm soát phiên bản (Git, GitHub).

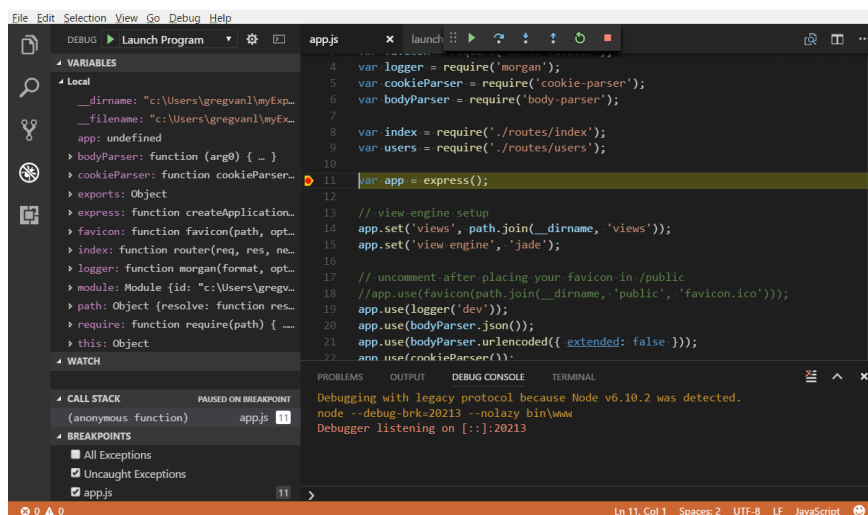
- Có kho tiện ích mở rộng phong phú, hỗ trợ biên dịch, kiểm thử và triển khai trực tiếp.
- **Ý nghĩa:** VS Code giúp tăng hiệu quả lập trình, kiểm thử và phối hợp nhóm trong phát triển hệ thống IoT, đồng thời thuận tiện khi mở rộng sang các nền tảng web hoặc ứng dụng di động.



Hình 3.7: Giao diện phần mềm Visual Studio Code

❖ NodeJS

- **Nhiệm vụ:** NodeJS là môi trường chạy phía máy chủ, xử lý yêu cầu từ người dùng, nhận dữ liệu từ thiết bị (ESP32, Arduino) và giao tiếp với cơ sở dữ liệu.
- **Chức năng:**
 - Xây dựng web server bất đồng bộ, tốc độ cao.
 - Hỗ trợ RESTful API và WebSocket để truyền dữ liệu thời gian thực.
 - Tương thích tốt với MongoDB, cho phép xử lý và lưu trữ dữ liệu linh hoạt.
- **Ý nghĩa:** NodeJS giúp hệ thống hoạt động ổn định, mở rộng dễ dàng và đảm bảo hiệu năng khi có nhiều truy cập đồng thời, phù hợp với mô hình bãi giữ xe thông minh thời gian thực.



Hình 3.8: Hướng dẫn Node.js trong Visual Studio Code

❖ MongoDB

- **Nhiệm vụ:** MongoDB là hệ quản trị cơ sở dữ liệu chính, lưu trữ toàn bộ dữ liệu liên quan đến người dùng, booking, nhận diện biển số và lịch sử ra/vào xe.
- **Chức năng:**
 - Lưu trữ dữ liệu linh hoạt theo mô hình NoSQL, tối ưu cho dữ liệu phi cấu trúc.
 - Hỗ trợ truy vấn nhanh và mở rộng quy mô khi tăng số lượng người dùng hoặc bãi xe.
 - Tích hợp tốt với NodeJS thông qua thư viện Mongoose.
- **Ý nghĩa:** MongoDB đảm bảo tính toàn vẹn, tốc độ và khả năng mở rộng dữ liệu cho hệ thống, giúp việc đồng bộ thông tin giữa thiết bị IoT và phần mềm quản lý diễn ra hiệu quả.



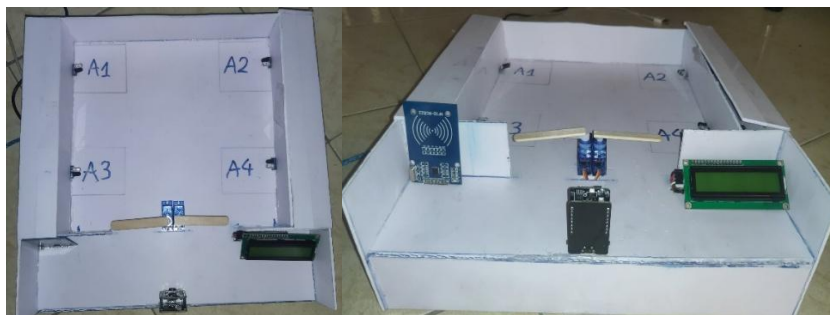
Hình 3.9: Hướng dẫn MongoDB trong Visual Studio Code

CHƯƠNG 4: KẾT QUẢ THỰC HIỆN VÀ KIỂM THỬ

4.1. Kết quả thực hiện

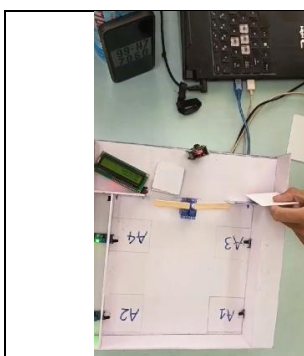
4.1.1. Phần cứng

- Thiết kế được sơ đồ khối và bố trí linh kiện một cách hợp lý và phù hợp với yêu cầu hệ thống.
- Các cảm biến IR hoạt động ổn định trong việc phát hiện vị trí đỗ còn trống hay đã có xe.
- Module RFID nhận dạng người dùng chính xác khi quét thẻ.
- Hệ thống hiển thị thông tin (số lượng vị trí đỗ còn trống và mức phí giữ xe mà khách hàng phải trả) trên LCD đầy đủ và rõ ràng.
- Servo barie được điều khiển tự động đóng/mở đúng thời điểm, đảm bảo an toàn và tiện lợi.
- Kết nối Arduino và Esp32 để truyền dữ liệu nội bộ.



Hình 4.1: Mô hình phần cứng

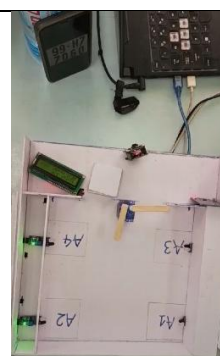
❖ Hình ảnh sau khi thực hiện:



Hình 4.2: Quét thẻ RFID



Hình 4.3: Camera chụp biển số xe



Hình 4.4: Rào chắn mở cho xe vào bãi

4.1.2. Phần mềm

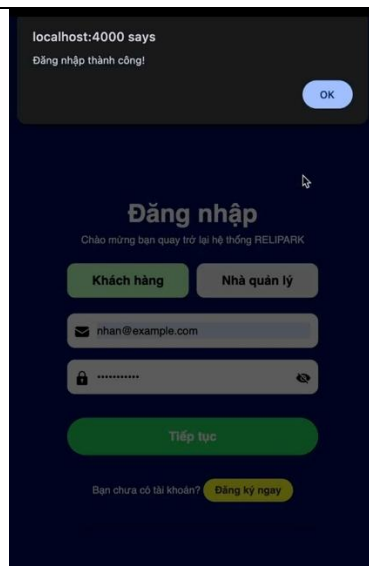
- Lập trình Arduino Uno đọc tín hiệu cảm biến và RFID.
- Viết chương trình điều khiển servo barie và hiển thị LCD.
- Thiếp lập cấu trúc dữ liệu lưu trạng thái chỗ đỗ, mã thẻ và thời gian.
- Gửi dữ liệu Arduino → ESP32 qua UART hoặc I2C.
- Lập trình ESP32 kết nối Wifi và giao tiếp với web server.
- Xây dựng web/app hiển thị trạng thái bãi đỗ và số tiền dự kiến.
- Đồng bộ dữ liệu thời gian thực giữ hệ thống và server.

❖ Giao diện của khách hàng:

 Đăng ký Tạo tài khoản mới để sử dụng dịch vụ RELIPARK Họ và Tên Email Số điện thoại Mật khẩu Biển số xe Đăng ký Bạn đã có tài khoản? Đăng nhập ngay	 Đăng nhập Chào mừng bạn quay trở lại hệ thống RELIPARK Khách hàng Nhà quản lý Email / Số điện thoại Mật khẩu Tiếp tục Bạn chưa có tài khoản? Đăng ký ngay
---	---

Hình 4.7: Giao diện đăng ký của khách hàng

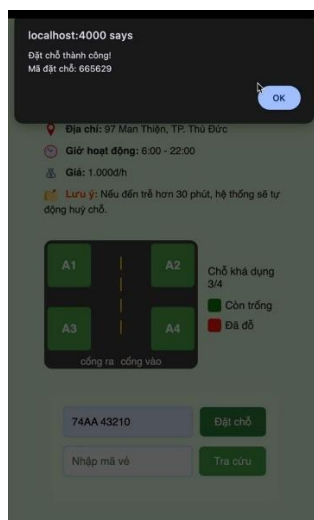
Hình 4.8: Giao diện đăng nhập của khách hàng



Hình 4.9: Giao diện đăng nhập thành công



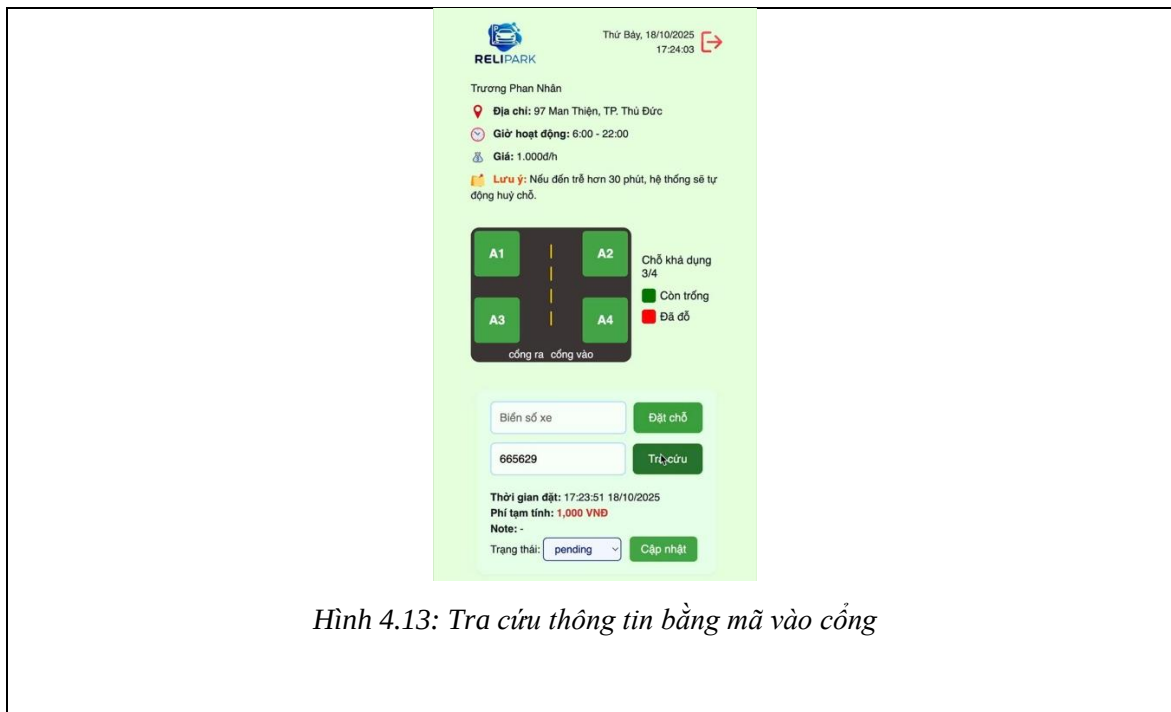
Hình 4.10: Giao diện đặt chỗ



Hình 4.11: Giao diện đặt chỗ thành công



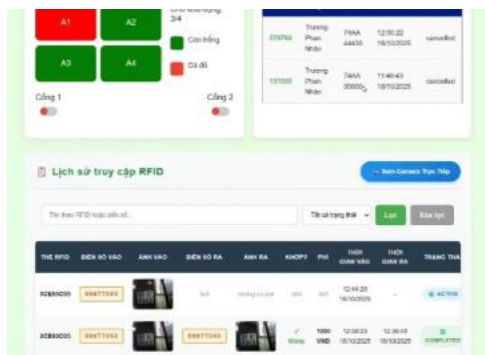
Hình 4.12: Vé đặt chỗ thành công



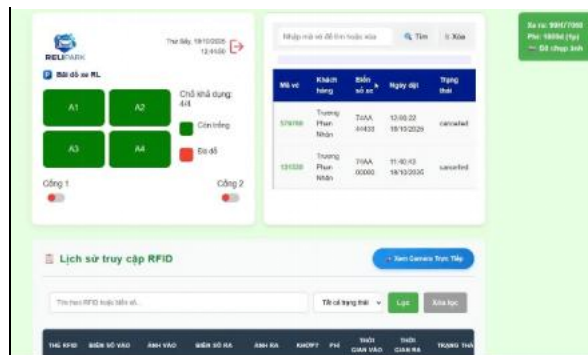
❖ Giao diện của nhà quản lý:



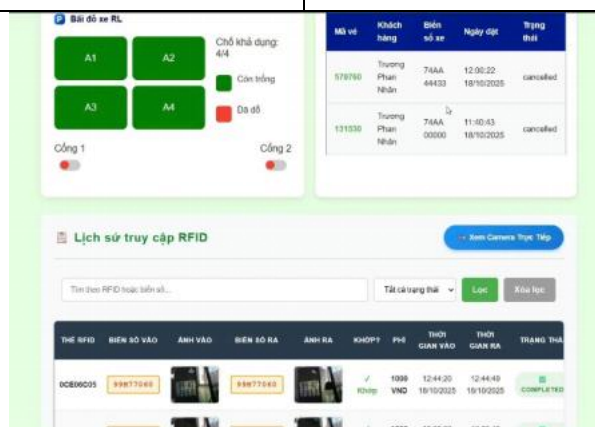
BÀI BÁO CÁO TỔNG KẾT NGHIÊN CỨU KHOA HỌC SINH VIÊN 2025-2026



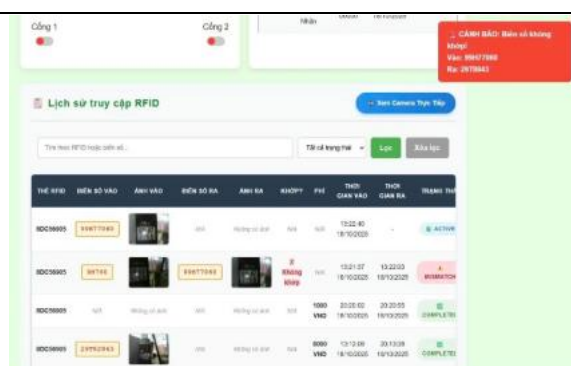
Hình 4.16: Biển số xe đã được lưu vào lịch sử truy cập



Hình 4.17: Chụp ảnh biển số khi xe rời bãi thành công



Hình 4.18: Biển số xe vào và rời bãi đã khớp nhau



Hình 4.19: Chụp ảnh biển số khi xe rời bãi thất bại



Hình 4.20: Biển số xe vào và rời bãi không khớp nhau

4.2. Kiểm thử

Trong quá trình kiểm thử và đánh giá, nhóm đã kiểm tra 3 trường hợp có thể xảy ra.

Khi khách hàng muốn tìm chỗ giữ xe, nhóm đặt mục tiêu ban đầu là khi khách hàng vào trang web điền đầy đủ thông tin và nhấn đăng ký thì hệ thống báo “đăng ký thành công”, khi đăng nhập (điền lại thông tin đã đăng ký) thành công thì hệ thống tiếp tục thông báo “đăng nhập thành công”. Về giao diện khách hàng, hệ thống phải hiển thị đúng số lượng chỗ khả dụng. Kết quả kiểm thử là hệ thống thông báo chính xác, số lượng chỗ trống khả dụng đã giảm đi 1 khi khách hàng đặt chỗ thành công và hệ thống tự động cập mã xác thực. Khi vào bãi, khách hàng xác thực mã, quét thẻ RFID, camera sẽ chụp, lưu lại biển số xe thì rào chắn tự động mở. Khi rời bãi, quét thẻ RFID để xác minh, đồng thời camera chụp lại biển số nhằm đối chiếu với dữ liệu vào bãi. Nếu đúng thông tin, hệ thống sẽ tự động tính phí và hiển thị số tiền lên LCD và trên web (khách hàng cũng có thể tra cứu trước đó để chuẩn bị tiền). Sau khi thanh toán thành công, rào chắn sẽ tự động mở, số lượng chỗ trống khả dụng cũng tăng lên 1. Nếu thông tin không tin không trùng khớp, rào chắn sẽ không tự động mở. Nhóm đánh giá hệ thống đã đạt đầy đủ các yêu cầu của trường hợp.

Khi khách hàng đến trực tiếp bãi mà chưa đặt chỗ trước, mục tiêu kiểm thử là xem hệ thống có hiển thị đúng số chỗ còn trống và cửa có mở hay không. Sau khi kiểm thử, nhóm nhận thấy rằng hệ thống vẫn hiển thị đúng số chỗ trống khả dụng trên LCD, cổng cũng sẽ tự động mở khi quét thẻ RFID, camera chụp, lưu lại biển số thành công.

Trường hợp khách hàng đã đăng ký và đặt chỗ. Nhóm đã kiểm tra xem nếu khách hàng đến bãi trễ sau khoảng thời gian được phép giữ chỗ (30 phút) thì số chỗ trống khả dụng hiển thị trên web và LCD có thay đổi hay không? Kết quả là thời gian giữ chỗ thực tế quy định là 30 phút nhưng hệ thống chỉ mô phỏng trong khoảng 30 giây và sau 30 giây đó nếu khách hàng chưa đến bãi số chỗ trống lúc này sẽ tăng lên 1. Như vậy trường hợp này cũng đã đạt yêu cầu nhưng do hệ thống đang trong quá trình thử nghiệm, chưa đưa vào thực tế nên thời gian giữ chỗ chỉ để 30 giây.

4.3. Nhận xét

Qua ba trường hợp kiểm thử, nhóm nhận thấy hệ thống đáp ứng đầy đủ mục tiêu đề tài, hoạt động ổn định và tự động. Các chức năng chính (đặt chỗ, quét RFID, nhận diện biển số, tính phí và mở rào chắn) đều vận hành chính xác và đồng bộ nếu kết nối Wi-Fi ổn định.

Tuy nhiên, trong một số trường hợp khi mạng Wi-Fi yếu hoặc mất kết nối tạm thời, dữ liệu có thể bị truyền chậm, gây trễ trong việc cập nhật số chỗ trống hoặc mở rào chắn.

Tổng thể, hệ thống đạt yêu cầu kiểm thử, hoạt động chính xác, có tính ứng dụng thực tế cao và độ ổn định phụ thuộc vào chất lượng kết nối mạng Wi-Fi — yếu tố cần được tối ưu khi triển khai thực tế.

CHƯƠNG 5: KẾT LUẬN

Qua quá trình nghiên cứu và thực hiện đề tài “Thiết kế và thi công mô hình bãi giữ xe thông minh có thể theo dõi, quản lý từ xa qua thiết bị di động”, nhóm đã hoàn thành một hệ thống có đầy đủ phần cứng và phần mềm, ứng dụng công nghệ IoT trong quản lý và điều hành bãi giữ xe. Hệ thống hoạt động dựa trên các mô-đun cảm biến hồng ngoại (LM393), đầu đọc RFID RC522, camera ESP32-CAM, vi điều khiển Arduino Uno R3 và ESP32, cho phép tự động hóa quá trình ra vào, nhận diện phương tiện tiện ích, hiển thị số lượng chỗ còn trống, cũng như đồng bộ hóa dữ liệu thời gian thực và hiển thị lên trang web. Trên nền tảng phần mềm, các công cụ như Arduino IDE, Visual Studio Code, NodeJS và MongoDB được sử dụng để thiết lập chương trình, thiết kế giao diện, xây dựng trang web và cơ sở dữ liệu. Hệ thống web hỗ trợ người dùng có thể đăng nhập, đặt chỗ trước, xem trạng thái bãi giữ xe, tra cứu phí giữ xe, cũng như giúp nhà quản lý theo dõi số lượng xe, lịch sử ra vào và trạng thái gửi theo thời gian thực.

Kết quả đạt được cho thấy mô hình hoạt động ổn định, các cảm biến phản hồi nhanh chóng, RFID nhận dạng thẻ chính xác và giao diện web thân thiện, dễ thao tác. Hệ thống đã đáp ứng mục tiêu đầu tiên là thiết kế và hoàn thành mô hình bãi giữ xe. Về phần mềm, xây dựng thành công web đặt chỗ trước giúp người dùng có thể đặt giữ chỗ và biết trước phí giữ xe phải trả nhằm tiết kiệm thời gian, sau khi khách hàng đăng ký và đặt chỗ thành công dữ liệu được lưu trữ ổn định trong cơ sở dữ liệu MongoDB, có thể truy xuất, phân tích và cập nhật dễ dàng. Ngoài ra, việc thêm chức năng chụp, nhận diện và lưu lại biển số xe cũng góp phần đảm bảo an ninh cho bãi giữ xe.

Tuy nhiên, mô hình vẫn tồn tại một số hạn chế nhất định. Cụ thể, hệ thống mới chỉ tương tác qua giao diện web, chưa xây dựng được app dùng trên thiết bị di động nên trải nghiệm của người dùng còn giới hạn. Kết nối Internet WiFi còn chưa ổn định, ảnh hưởng đến tốc độ thực thi dữ liệu. Việc theo dõi và quản lý từ xa chưa được hoàn thiện và khả năng định vị chỉ đường đến vị trí đỗ chưa được thực hiện. Ngoài ra, mô hình hiện tại ở quy mô nhỏ, chưa đáp ứng được những yêu cầu vận hành thực tế của các bãi xe lớn.

Dựa trên kết quả đạt được, nhóm đề xuất phát triển trong tương lai theo các hướng sau: Xây dựng ứng dụng di động dành riêng cho người dùng giúp đặt chỗ, thanh toán và kiểm tra trạng thái xe nhanh hơn. Mở rộng quy mô mô hình để phù hợp với các loại xe lớn và lượng xe nhiều hơn. Hệ thống cần tích hợp mã QR cho phép người dùng quét để mở cổng và thanh toán tự động, giúp rút ngắn thời gian xử lý, giảm nhân lực và nâng cao trải nghiệm người dùng. Đồng thời phát triển thêm chức năng chỉ đường tới bãi giữ xe và vị trí đỗ còn trống. Để thông minh hơn, ta cần tích hợp thêm cảm biến xác định chính xác vị trí xe đang đỗ và chỉ đường tới vị trí đó trên ứng dụng di động, nhờ đó, người dùng có thể dễ dàng tra cứu và xác định chính xác vị trí xe của mình khi ra về. Hướng phát triển này sẽ giúp hệ thống trở nên hoàn thiện, an toàn, tiện lợi và phù hợp hơn với xu hướng đô thị thông minh và chuyển đổi số trong giao thông hiện đại.

TÀI LIỆU THAM KHẢO

- [1] Châu Tuấn, “Ùn tắc giao thông ở TP.HCM: Phải dừng từ 30 phút, dài 200m mới tính?”, *Báo tuổi trẻ online*, 2025. [Online]. Available at: <https://byvn.net/wjRA>
- [2] VinFast, “5 bãi đỗ xe thông minh ở Hà Nội chất lượng tốt hiện nay”, Hà Nội (Việt Nam), 2021. [Online]. Available at: <https://byvn.net/Oa5H>
- [3] Bilparking, “Hệ thống bãi đỗ xe tự động xếp hình”, Việt Nam. [Online]. Available at: <https://byvn.net/mMvi>
- [4] Greensmart, “Bãi giữ xe ô tô thông minh giúp tiết kiệm không gian và chi phí”, Việt Nam, 2024. [Online]. Available at: <https://byvn.net/QcIN>
- [5] Roberto Minerva, Abyi Biru, Domenico Rotondi, *Towards a definition of the Internet of Things (IoT)*. Italia, 2015.
- [6] Serpanos D. N. và Wolf M., *Internet-of-Things (IoT) Systems: Architectures, Algorithms, Methodologies*. Đức, 2018.
- [7] “Electronic Components Datasheet Search”. [Online]. Available at: <https://www.alldatasheet.com>
- [8] arduinokit, “Chuẩn giao tiếp SPI là gì? Giao thức SPI trong Arduino”, Việt Nam, 2023. [Online]. Available at: <https://byvn.net/pdic>
- [9] arduinokit, “Khái niệm cơ bản về chuẩn giao tiếp I2C trong Arduino”, Việt Nam, 2023. [Online]. Available at: <https://byvn.net/y3PL>
- [10] BKAIL, “Tìm hiểu về truyền thông UART: khái niệm, nguyên lý và ứng dụng”, Việt Nam. [Online]. Available at: <https://byvn.net/2lXB>
- [11] IETF, “The WebSocket Protocol”. 2011.
- [12] IETF, “Hypertext Transfer Protocol”. 1999.

PHỤ LỤC CODE

➤ **Link:** <https://github.com/NhanTr/parkingFinal>

❖ **esp32.ino:**

▪ **Hàm khởi tạo chính:**

```
//Khởi tạo toàn bộ hệ thống
void setup() {
    Serial.begin(9600);

    Wire.begin(21, 22);
    lcd.init();
    lcd.backlight();
    displayMessage("Smart Parking", "RFID System", 2000);

    initializeWiFi();      //Kết nối WiFi
    timeClient.begin();
    initializeParkingData(); //Khởi tạo giá trị mặc định cho dữ liệu bãi đỗ
    initializeWebSocket();  //Thiết lập kết nối WebSocket với server

    Serial.println("RFID System started");
    sendModeToArduino();
    updateDisplay();
}
```


▪ Vòng lặp chính

```
void loop() {  
    if (WiFi.status() == WL_CONNECTED) {  
        wsClient.loop();  
        sendKeepAlive();  
    } else {  
        WiFi.begin(ssid, password);  
        wsConnected = false;  
        delay(1000);  
    }  
  
    timeClient.update();  
  
    if (Serial.available()) {  
        String data = Serial.readStringUntil('\n');  
        parseArduinoData(data);  
    }  
  
    checkRfidResponseTimeout();  
    checkServerStatusTimeout();  
  
    static unsigned long lastDisplayUpdate = 0;  
    if (millis() - lastDisplayUpdate > 200) {  
        updateDisplay();  
        lastDisplayUpdate = millis();  
    }  
  
    static unsigned long lastPeriodicUpdate = 0;
```

```
if (wsConnected && millis() - lastPeriodicUpdate > 2000) {  
    sendStatusResponse();  
    lastPeriodicUpdate = millis();  
}  
  
delay(100);  
}
```

▪ **Xử lý sự kiện**

```
void parseArduinoData(String data) {  
    data.trim();  
  
    if (data.length() == 0) return;  
  
    if (data.startsWith("RFID_DATABASE:")) {  
        handleRfidDatabase(data);  
    } else if (data.startsWith("HISTORY:")) {  
        handleHistoryMessage(data);  
    } else if (data.startsWith("VEHICLE_ENTRY:") || data.startsWith("VEHICLE_EXIT:")) {  
        Serial.println(data);  
    } else if (data.startsWith("{}")) {  
        handleJsonData(data);  
    } else if (data == "OPEN_ENTRY_GATE" || data == "OPEN_EXIT_GATE") {  
        handleGateControl(data);  
    } else if (data.startsWith("MODE_CHANGED:")) {  
        Serial.println("Mode confirmed: " + data.substring(13));  
    }  
}
```

```
}  
}  
  
void checkRfidResponseTimeout() {  
    if (!pendingRfidRequest.waitingForResponse) return;  
    if (millis() - pendingRfidRequest.requestTime > RFID_RESPONSE_TIMEOUT) {  
        Serial.println("DATABASE_RESPONSE:ERROR");  
        pendingRfidRequest.waitingForResponse = false;  
    }  
}  
  
void checkServerStatusTimeout() {  
    if (useServerStatus && (millis() - lastServerStatusUpdate >  
SERVER_STATUS_TIMEOUT)) {  
        useServerStatus = false;  
        forceDisplayUpdate = true;  
    }  
}  
  
void updateDisplay() {  
    lcd.clear();  
    lcd.setCursor(0, 0);  
  
    int availableCount;  
    int totalCount;  
    if (wsConnected && useServerStatus && serverParkingData.hasData) {  
        availableCount = serverParkingData.availableSlots;  
        totalCount = serverParkingData.totalSlots;  
    } else {  
        availableCount = parkingData.availableSlots;  
        totalCount = parkingData.totalSlots;  
    }  
}
```

```
lcd.print("Slots: " + String(availableCount) + "/" + String(totalCount));

if (!wsConnected) lcd.print(" (!)");
lcd.setCursor(0, 1);
if (availableCount > 0) {
    lcd.print("RFID Ready: ");
    // TRẠNG THÁI SLOT: Luôn theo local data
    for (int i = 0; i < 4; i++) {
        lcd.print(parkingData.slots[i] ? "O" : "X");
    }
} else {
    lcd.print("PARKING FULL");
}
}
```

```
void sendStatusResponse() {
    if (!wsConnected) return;

    DynamicJsonDocument statusDoc(1024);
    ParkingData* data = useServerStatus && serverParkingData.hasData ?
        (ParkingData*)&serverParkingData : &parkingData;

    statusDoc["availableSlots"] = data->availableSlots;
    statusDoc["totalSlots"] = data->totalSlots;
    statusDoc["entryGateOpen"] = data->entryGateOpen;
    statusDoc["exitGateOpen"] = data->exitGateOpen;
    statusDoc["lastUpdate"] = data->lastUpdated;
    statusDoc["isAdminMode"] = data->isAdminMode;
    statusDoc["isRFIDMode"] = true;

    JsonArray slots = statusDoc.createNestedArray("slots");
    for (int i = 0; i < 4; i++) {
        slots.add(data->slots[i]);
    }
}
```

```
}  
String statusJson;  
  
serializeJson(statusDoc, statusJson);  
  
wsClient.sendTXT("{\"type\":\"status\",\"data\":\"" + statusJson + "\"}");  
}
```

```
void handleRfidDatabase(String data) {  
    String dbData = data.substring(14);  
    int indices[4];  
  
    findCommaIndices(dbData, indices, 4);  
  
    if (indices[3] != -1) {  
        String event = dbData.substring(0, indices[0]);  
        String rfidCode = dbData.substring(indices[0] + 1, indices[1]);  
        String action = dbData.substring(indices[1] + 1, indices[2]);  
        String servoType = dbData.substring(indices[2] + 1, indices[3]);  
  
        displayMessage("Welcome!", rfidCode, 1500);  
        displayMessage("Processing...", "Please wait", 0);  
        sendRfidAccessToDatabase(rfidCode, action, servoType);  
    }  
}
```

```
void sendRfidAccessToDatabase(String rfidCode, String action, String servoType) {  
    if (!wsConnected) {  
        Serial.println("DATABASE_RESPONSE:ERROR");  
        displayMessage("NO CONNECTION", "Entry allowed", 2000);  
        return;  
    }  
}
```

```
setPendingRequest(rfidCode, action);

DynamicJsonDocument doc(512);
doc["type"] = "rfid_access";
doc["data"]["rfidCode"] = rfidCode;
doc["data"]["action"] = action;
doc["data"]["servoType"] = servoType;
doc["data"]["timestamp"] = getISOTimestamp();
doc["data"]["deviceId"] = "ESP32_PARKING_SYSTEM";
doc["data"]["method"] = "RFID";

String slotInfo = "Unknown";
if (servoType == "ENTRY_GATE") {
    for (int i = 0; i < 4; i++) {
        if (useServerStatus && serverParkingData.hasData) {
            if (serverParkingData.slots[i] == 0) {
                slotInfo = "A" + String(i + 1);
                break;
            }
        } else {
            if (!parkingData.slots[i]) {
                slotInfo = "A" + String(i + 1);
                break;
            }
        }
    }
}
```

```
doc["data"]["slotUsed"] = slotInfo;

String rfidJson;

serializeJson(doc, rfidJson);

wsClient.sendTXT(rfidJson);
}
```

```
void addHistory(String event, String idCode, String action,
                String method, String servoType, String triggerSource) {
    if (historycount < 20) {
        setHistoryEntry(historycount, event, idCode, action, method, servoType, triggerSource);
        historycount++;
    } else {
        shiftHistoryUp();
        setHistoryEntry(19, event, idCode, action, method, servoType, triggerSource);
    }
    if (wsConnected) {
        sendHistoryUpdate();
    }
}
```

```
void notifyServoActivation(String servoType, String triggerMethod, String triggerCode,
String triggerSource) {

    if (!wsConnected) return;

    if (servoType == "PENDING") {
        servoType = parkingData.entryGateOpen ? "ENTRY_GATE" :
            (parkingData.exitGateOpen ? "EXIT_GATE" : "N/A");
    }
    DynamicJsonDocument doc(512);
```

```
doc["type"] = "servo_activation";
doc["servoType"] = servoType;
doc["triggerMethod"] = "RFID";
doc["triggerCode"] = triggerCode;
doc["triggerSource"] = triggerSource;
doc["timestamp"] = timeClient.getFormattedTime();

String json;
serializeJson(doc, json);
wsClient.sendTXT(json);
}

void sendHistoryUpdate() {
  if (!wsConnected) return;
  DynamicJsonDocument historyDoc(3072);
  JsonArray historyArray = historyDoc.createNestedArray("history");

  for (int i = 0; i < historycount; i++) {
    JsonObject entry = historyArray.createNestedObject();
    entry["event"] = history[i].event;
    entry["idCode"] = history[i].idCode;
    entry["action"] = history[i].action;
    entry["method"] = "RFID";
    entry["servoType"] = history[i].servoType;
    entry["triggerSource"] = history[i].triggerSource;
    entry["timestamp"] = history[i].timestamp;
  }

  String historyJson;
```



```
serializeJson(historyDoc, historyJson);  
wsClient.sendTXT("{\"type\":\"history\\\", \"data\\\": \" + historyJson + \"}\"");  
}  
  
void displaySlotStatus(ParkingData* data) {  
    for (int i = 0; i < 4; i++) {  
        if (useServerStatus && serverParkingData.hasData) {  
            lcd.print(serverParkingData.slots[i] == 1 ? "O" :  
                serverParkingData.slots[i] == 2 ? "R" : "X");  
        } else {  
            lcd.print(data->slots[i] ? "O" : "X");  
        }  
    }  
}  
}
```

❖ **esp32_cam.ino:**

```
void captureAndSendImage(String rfidCode, String action, String gateType) {  
    Serial.println("=== CAPTURE START ===");  
    Serial.print("RFID: ");  
    Serial.println(rfidCode);  
    Serial.print("Action: ");  
    Serial.println(action);  
  
    const int MAX_RETRIES = 3;  
    const int NUM_IMAGES = 6; // Capture 6 images  
    camera_fb_t * fb = NULL;
```

```
// Turn on flash LED
digitalWrite(FLASH_LED_PIN, HIGH);
Serial.println("Flash LED ON");
delay(150); // Stabilize lighting

// Set camera to higher quality for capture
sensor_t * s = esp_camera_sensor_get();
s->set_framesize(s, FRAMESIZE_UXGA); // 1600x1200
s->set_quality(s, 10);

// Capture 6 images, only keep the 6th
for (int image_num = 1; image_num <= NUM_IMAGES; image_num++) {
    Serial.print("Capturing image ");
    Serial.print(image_num);
    Serial.print("/");
    Serial.println(NUM_IMAGES);

    // Attempt capture with retries
    for (int attempt = 1; attempt <= MAX_RETRIES; attempt++) {
        Serial.print("Attempt ");
        Serial.print(attempt);
        Serial.print("/");
        Serial.println(MAX_RETRIES);

        fb = esp_camera_fb_get();
```

```
if (fb) {
    Serial.println("Capture successful!");
    // If this is not the 6th image, return the buffer and continue
    if (image_num < NUM_IMAGES) {
        esp_camera_fb_return(fb);
        fb = NULL;
    }
    break; // Success, move to next image
} else {
    Serial.print("Capture failed on attempt ");
    Serial.println(attempt);

    if (attempt < MAX_RETRIES) {
        Serial.println("Retrying...");
        delay(500); // Wait before retry
    }
}

if (!fb && image_num == NUM_IMAGES) {
    digitalWrite(FLASH_LED_PIN, LOW);
    s->set_framesize(s, FRAMESIZE_SVGA);
    s->set_quality(s, 12);
    sendCaptureError(rfidCode, action, "Camera capture failed after retries");
    return;
}
```

```
    delay(50); // Small delay between captures
}

digitalWrite(FLASH_LED_PIN, LOW); // Tắt đèn

s->set_framesize(s, FRAMESIZE_SVGA); // Restore streaming quality
s->set_quality(s, 12);

if (fb) {    // Process the 6th image if captured
    Serial.print("Final image captured: ");
    Serial.print(fb->len);
    Serial.println(" bytes");

    if (fb->len < 1000) { // Validate image size
        Serial.println("Image too small, likely corrupted");
        esp_camera_fb_return(fb);
        sendCaptureError(rfidCode, action, "Image corrupted");
        return;
    }

    // Convert to Base64
    String base64Image = base64::encode(fb->buf, fb->len);
    Serial.print("Base64 length: ");
    Serial.println(base64Image.length());
```

```
sendImageToServer(rfidCode, action, gateType, base64Image); //sending to sv

esp_camera_fb_return(fb); // Return frame buffer
}
}
```

```
void sendImageToServer(String rfidCode, String action, String gateType, String
base64Image) {

    const size_t CHUNK_SIZE = 8000; // 8KB chunks
    size_t totalLength = base64Image.length();
    size_t chunks = (totalLength + CHUNK_SIZE - 1) / CHUNK_SIZE;
    Serial.printf("Sending image in %d chunks\n", chunks);

    for (size_t i = 0; i < chunks; i++) {
        size_t start = i * CHUNK_SIZE;
        size_t end = min(start + CHUNK_SIZE, totalLength);
        String chunk = base64Image.substring(start, end);

        DynamicJsonDocument doc(10000);
        doc["type"] = "camera_image";
        doc["data"]["rfidCode"] = rfidCode;
        doc["data"]["action"] = action;
        doc["data"]["gateType"] = gateType;
        doc["data"]["chunk"] = i;
        doc["data"]["totalChunks"] = chunks;
        doc["data"]["imageData"] = chunk;
```

```
String msg;
serializeJson(doc, msg);
wsClient.sendTXT(msg);

Serial.print("Sent chunk ");
Serial.print(i + 1);
Serial.print("/");
Serial.print(chunks);
Serial.print(" (");
Serial.print((i + 1) * 100 / chunks);
Serial.println("%)");

delay(50); // Small delay between chunks
}
}

void sendStreamFrame() {
    if (!streamingEnabled || !wsConnected) return;

    unsigned long currentTime = millis();
    if (currentTime - lastStreamFrame < STREAM_INTERVAL) return;

    camera_fb_t * fb = esp_camera_fb_get();
    if (!fb) {
        return;
    }
}
```

```
String base64Frame = base64::encode(fb->buf, fb->len);

DynamicJsonDocument doc(base64Frame.length() + 200);
doc["type"] = "stream_frame";
doc["data"] = base64Frame;
doc["timestamp"] = currentTime;

String msg;
serializeJson(doc, msg);
wsClient.sendTXT(msg);

esp_camera_fb_return(fb);
lastStreamFrame = currentTime;
}
```

❖ **arduino.ino:**

```
void loop() {
  if (Serial.available()) {
    String command = Serial.readStringUntil('\n');
    command.trim();
    processSerialCommand(command);
  }
  updateParkingStatus();
  checkRFID();
  checkGateDelay();
}
```

```
checkGateTimeout();
checkDatabaseResponseStatus();

if (millis() - lastUpdate > 1000) {
    sendDataToESP32();
    lastUpdate = millis();
}

if (millis() - lastHeartbeat > 10000) {
    lastHeartbeat = millis();
}
delay(50);
}

void checkRFID() {
    if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) {
        return;
    }

    char cardID[9];
    memset(cardID, 0, 9);
    for (byte i = 0; i < rfid.uid.size && i < 4; i++) {
        sprintf(cardID + i*2, "%02X", rfid.uid.uidByte[i]);
    }
    cardID[8] = '\0';

    Serial.print(F("RFID detected: "));
```



```
Serial.println(cardID);

rfid.PICC_HaltA();

if (isAuthorizedCard(cardID)) {
    int vehicleIndex = findVehicleByID(cardID);

    if (vehicleIndex != -1) {
        handleExit(cardID);
    } else if (availableSlots > 0) {
        handleEntry(cardID);
    } else {
        Serial.println(F("Parking FULL - Cannot enter"));
    }
} else {
    Serial.print(F("Unauthorized RFID: "));
    Serial.println(cardID);
}
}
```

```
void updateParkingStatus() {
    parkingSlots[0] = (digitalRead(IR_SLOT1) == LOW);
    parkingSlots[1] = (digitalRead(IR_SLOT2) == LOW);
    parkingSlots[2] = (digitalRead(IR_SLOT3) == LOW);
    parkingSlots[3] = (digitalRead(IR_SLOT4) == LOW);
}
```

```
void checkGateDelay() {  
    unsigned long currentTime = millis();  
    if (entryGateWaitingToOpen) {  
        if (currentTime - gateDelayStartTime >= GATE_DELAY_DURATION) {  
            if (waitingForDatabaseResponse) {  
                return;  
            }  
            if (!dbRequest.responseSuccess) {  
                Serial.println(F("Database error, gate not opened"));  
                entryGateWaitingToOpen = false;  
                return;  
            }  
            if (availableSlots <= 0) {  
                Serial.println(F("Parking FULL"));  
                entryGateWaitingToOpen = false;  
                return;  
            }  
            servoEntry.write(90);  
            entryGateOpen = true;  
            gateOpenTime = millis();  
            entryGateWaitingToOpen = false;  
            Serial.println(F("Entry gate opened after 5s delay"));  
        }  
    }  
    if (exitGateWaitingToOpen) {  
        if (currentTime - gateDelayStartTime >= GATE_DELAY_DURATION) {  
            if (waitingForDatabaseResponse) {
```

```
Serial.println(F("Waiting for database response"));
return;
}
if (!dbRequest.responseSuccess) {
    Serial.println(F("Database error"));
    exitGateWaitingToOpen = false;
    return;
}
servoExit.write(90);
exitGateOpen = true;
gateOpenTime = millis();
exitGateWaitingToOpen = false;
}
}
}
```

```
void checkGateTimeout() {
    unsigned long currentTime = millis();
    if (entryGateOpen && (currentTime - gateOpenTime > GATE_OPEN_DURATION)) {
        servoEntry.write(0);
        entryGateOpen = false;
        Serial.println(F("Entry gate closed due to timeout"));
    }
    if (exitGateOpen && (currentTime - gateOpenTime > GATE_OPEN_DURATION)) {
        servoExit.write(0);
        exitGateOpen = false;
        Serial.println(F("Exit gate closed due to timeout"));
    }
}
```

}